Bounded Rescheduling for Flexible Job Shops via Spatiotemporal Reservation Closure

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

In flexible job shop scheduling that captures processing order with operation precedence and records corridor occupancies with time-window reservations, the standard way to delineate the rescheduling scope after a disturbance is still to identify the affected operations and reschedule them together with their successors. When a corridor becomes impassable or slower during some time window, however, no operation becomes unexecutable. This approach then returns an empty scope and concludes that no rescheduling is needed, although the corridor occupancies inside the disturbed interval are already invalid, the original schedule is infeasible, and the resulting waits propagate to downstream vehicles through yielding relations. The gap lies not in detection but in the fact that the operation-precedence graph contains no yielding relations among corridor occupancies.

We define the rescheduling scope on corridor occupancies and their dependencies and propose the spatiotemporal reservation closure (STRC), whose result is called the reservation impact set. Starting from the corridor reservations whose occupancy intervals overlap the disturbance window, STRC takes the transitive closure along four kinds of edges: yielding on the same corridor, and the next occupancy of the same vehicle, of the same job, and of the same machine. Occupancies inside the set may be rewritten; those outside it keep the original plan. Once the dependency edges are fixed, membership of an occupancy in the set is uniquely determined and does not depend on how the rerouting algorithm is implemented. Bounded rescheduling then repairs only inside the set, in two levels: the original vehicles are first rerouted, and if the schedule remains infeasible, the rewritable scope is enlarged in a prescribed order.

Experiments cover 5 instances × 10 random seeds, i.e., 50 pairs. Under corridor blockage, the scope delineated on the operation-precedence graph is empty on 50/50 pairs, while the original schedule is infeasible on all of them. With the rerouting procedure unchanged and only the scope object replaced, the feasibility rate rises from 0/50 to 50/50. The impact set is closed under outgoing edges (no dependency edge leads from inside to outside) on 50/50 pairs, and the corridor, interval, and vehicle of every occupancy outside the set are unchanged on 50/50 pairs. Replacing the rewritable set with all occupancies still unfinished at the decision time, with everything else unchanged, leaves runtime and makespan almost unchanged but raises the rewrite fraction from 0.540 to 0.604 (per cell 31/18/1; two-sided Wilcoxon signed-rank test, $p = 8.8 \times 10^{-7}$). Compared with warm-started global rescheduling that likewise keeps finished occupancies, bounded repair is two to three orders of magnitude faster, but its makespan is worse at every budget level, and no crossover appears as the budget is relaxed.

The main effect comes from replacing the scope object: switching from the operation-precedence graph to the reservation impact set restores feasibility with the rerouting procedure unchanged. A secondary effect comes from the impact set being smaller than the set of all unfinished occupancies, i.e., fewer rewrites; since the former already covers about 92% of the latter, this benefit vanishes

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

as the two sets converge. Rerouting inside the set, keeping the plan outside it, and enlarging the scope after failure follow existing practice; the contribution of this paper lies in the scope object.

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → *Industry and manufacturing*; • **Computing methodologies** → *Planning and scheduling*.

Additional Key Words and Phrases: flexible job shop scheduling, bounded rescheduling, conflict-free AGV routing, spatiotemporal reservation table, spatiotemporal reservation closure, corridor blockage

ACM Reference Format:
Qi Yu and Wei Li. 2026. Bounded Rescheduling for Flexible Job Shops via Spatiotemporal Reservation Closure. 1, 1 (October 2026), 42 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

1.1 Background and motivation

Flexible job shop scheduling with transportation has long rested on a simplifying assumption: the time to move a job between two machines is a constant that can be read from a matrix indexed by location pairs. The assumption implicitly holds that vehicles never compete for the same corridor segment, so that travel time depends only on the distance between two points rather than on routing decisions. Once a finite fleet shares a sparse corridor network and reserves contested segments with time windows, travel time is no longer fixed data but the outcome of two decisions: the path each vehicle takes and the order in which vehicles occupy each segment. One vehicle's yielding delays another vehicle's arrival and, in turn, the start of downstream operations and their transport occupancies. A feasible schedule must then fix the operation assignment, the processing order, and a reservation table that records corridor occupancies; a disturbance may make an operation unexecutable, or it may invalidate only corridor occupancies without changing any operation assignment.

Studies that build the schedule once before execution and decide machine assignment jointly with conflict-free paths have shown that, when an operation can be processed on several machines, contention among vehicles in the corridors changes which machine is preferable: if the segment leading to a faster machine is occupied, a slower but more reachable machine may be the better choice. Candidate schedules must therefore be evaluated with time-window routing that accounts for waiting, and the search must query conflicts on the reservation table instead of relying on constant travel times [9, 17, 20]. These works answer how to build a high-quality conflict-free schedule before execution, when information is complete. Once the same shop has executed up to time t_{now} and a disturbance occurs, the questions become different: which corridor occupancies are invalidated, which occupancies must be rewritten to restore feasibility, and whether recovery can be completed within milliseconds without restarting a population-based search.

1.2 The gap the operation-precedence graph cannot capture

In dynamic scheduling of flexible job shops with transportation, the rescheduling scope is usually delineated on the operation-precedence graph. Specifically, the operations that can no longer be executed as planned because of an equipment failure serve as seeds; the subsequent operations of the same job and the operations sequenced after them on the same machine are added to the scope, or the expansion stops after at most θ steps from the seeds on the graph; only the operations in the scope are then rescheduled. Two dynamic scheduling studies with AGV transport and with machine failures or rush orders as disturbances delineate the scope in this way. Their recovery scopes are, respectively, "the operations not yet processed after the failure" [24] and "the operations and delivery tasks classified

by disturbance impact into retained, continued, and reconstructed" [19] (the latter studies a hybrid flow shop with lot streaming whose disturbances include vehicle failures, but its model has no reservation table). This practice can be traced back to affected-operation rescheduling (AOR) [19, 24]. In what follows, a baseline that delineates the scope on the operation-precedence graph represents this practice (Section 2.1).

This way of delineating the scope is often reasonable when the disturbance itself is an operation event, e.g., a machine failure that prevents several operations from being executed under the original assignment. When the schedule records corridor occupancies with time windows, however, there is another class of disturbances: those that make a corridor impassable for some period without changing any operation assignment. A typical example is corridor blockage, in which a corridor is closed to traffic during $[t_a, t_b)$. A blockage modifies neither machine assignment nor processing order, so no operation on the operation-precedence graph is directly affected; the scope delineated by this method is empty, and the method concludes that no rescheduling is needed. The operation-precedence graph thus cannot capture the corridor occupancies that must be recovered: planned occupancies that overlap the blockage interval conflict with the closure and are no longer feasible; a vehicle that consequently waits or detours then blocks subsequent vehicles through yielding relations, so the invalidation propagates across the reservation table.

"Cannot capture" here means that, by definition, the operation-precedence graph contains no yielding relations among corridor occupancies and therefore cannot express how occupancy invalidation propagates between vehicles. If transport times are still taken from a constant matrix indexed by location pairs, corridors are not contested resources and no reservation table exists, so this kind of invalidation cannot even be expressed in the model; the gap therefore arises only in settings that record corridor occupancies with time windows.

Accordingly, we distinguish two classes of disturbances.

- **Class A** (acting on the operation-precedence graph): robotic-arm failures, significant deviations in processing time, rush-order insertions, and *vehicle failures*. These disturbances prevent some operations from being executed as planned, so the seed operation set used to delineate the rescheduling scope is nonempty; scoping on the operation-precedence graph and scoping on the reservation impact set both yield nonempty scopes, so this class is not the main object for testing the scope mismatch. The reason for placing vehicle failures in Class A is given in Section 3.3: although a vehicle failure changes no operation assignment, it makes the operations carried by that vehicle unexecutable, so the mapping from a vehicle to the operations it carries still yields a nonempty seed operation set on the operation-precedence graph.
- **Class B** (acting only on the reservation table): corridor blockage, corridor slowdown, temporary revocation of local right of way, and the like. These disturbances make no operation unexecutable, so the seed operation set on the operation-precedence graph is empty; the set of corridor occupancies that overlap the disturbed interval is nonempty, and so is the reservation impact set obtained by taking them as seeds.

Class B disturbances are a problem-level counterexample to scoping on the operation-precedence graph: when a model captures processing order with the operation-precedence graph and records corridor occupancies with time windows, a rescheduling scope delineated only on the operation-precedence graph cannot represent disturbances such as corridor blockage. The two dynamic scheduling studies with AGVs cited above do not consider corridor contention among vehicles (Section 2.4), so the operation-precedence graph is the only scope object available to them; the mismatch arises only when both kinds of objects are present (Section 2.5). Figure 1 illustrates the gap. On the left, the operation-precedence graph contains no seed operation, so no rescheduling is deemed necessary; on the right, the occupancies on

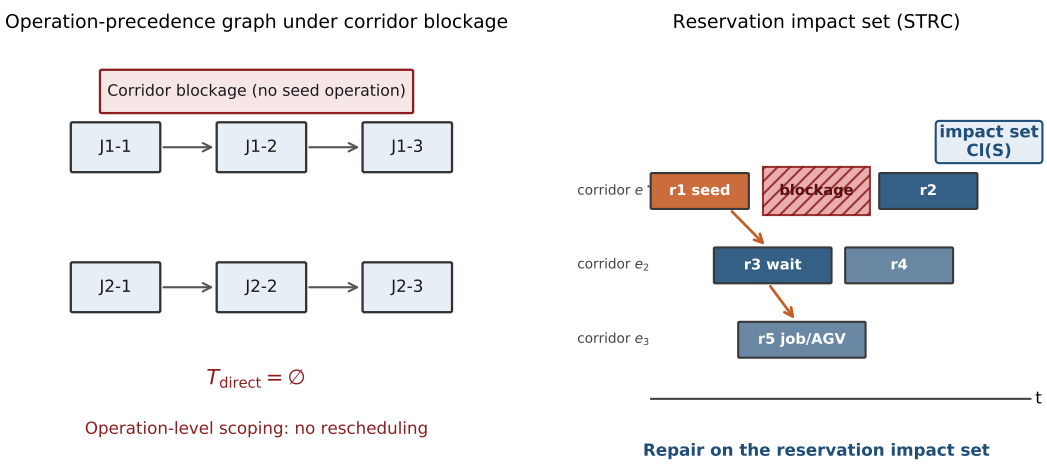


Fig. 1. Motivating example. Left: a corridor blockage makes no operation or machine unexecutable as planned, so the operation-precedence graph contains no seed operation. Right: under the same disturbance, the corridor occupancies that overlap the blockage interval become seeds and grow into a reservation impact set along yielding and succession relations.

the reservation table that overlap the blockage interval become seeds and grow into a reservation impact set along yielding and succession relations.

1.3 Claims

This paper makes three claims.

First, when a flexible job shop guarantees conflict-free vehicle movement through time-window reservations, the rescheduling scope after a disturbance should be defined on the reservation dependency graph formed by corridor occupancies and their yielding relations, not only on the operation-precedence graph.

Second, the scope is delineated by the spatiotemporal reservation closure (STRC), whose result is called the reservation impact set. Starting from the corridor occupancies that overlap the disturbance window and are therefore invalid, it follows yielding, the next occupancy of the same vehicle, the occupancy of the next operation of the same job, and the occupancy of the next operation on the same machine, and includes in the rescheduling scope every corridor occupancy that must, directly or indirectly, be discarded and rewritten. Once these dependencies are fixed, no occupancy outside the impact set is a one-step successor of an occupancy inside it along these relations, so "which occupancies to rewrite" corresponds to a verifiable set rather than a manually chosen search radius.

Third, once the impact set is obtained, the original vehicles are rerouted inside the set while occupancies outside it are kept; if no feasible schedule results, the rewritable scope is enlarged in the order of the same job, the same vehicle, and finally all unfinished occupancies. For Class A failures, reassigning vehicles and machines is additionally allowed, but the processing order is still kept. These steps follow existing practice and serve to act on the impact set and restore a feasible schedule.

The three claims share one stance: **the reservation impact set is a measurement, not a decision.** We do not use yielding relations among occupancies to guide the search toward different assignments or paths, because that would be

schedule improvement: removing one corridor wait often makes another constraint the new bottleneck, so the gain in makespan is easily offset. Instead, we use the same yielding relations to delineate which occupancies are already invalid. Once the dependencies are fixed, whether an occupancy belongs to the impact set depends only on whether it is reachable from the seeds along these relations; the result is unique and does not change with whichever constraint subsequently becomes the bottleneck.

Which dependency edges to include is a modeling choice; once the edge set is chosen, the closure of the impact set under outgoing edges (no occupancy in the set has a dependency edge leading outside it) follows from the definition (Proposition 2). The impact set is not necessarily the minimal set of occupancies needed to restore feasibility, because the minimal set depends on the permitted repair actions, whereas the definition of the impact set is not tied to any particular repair method (Remark 2).

Compared with global rescheduling that starts from the original schedule and searches again within a time limit, the reservation impact set addresses a different question: when an executable plan must be produced quickly, can feasibility be restored while confining rewrites to the impact set? Under the conditions of Proposition 3, after repair the corridor, interval, and vehicle of every corridor occupancy outside the impact set coincide with the original plan. Compared with global rescheduling that does not rewrite occupancies already finished at the decision time, bounded repair yields a worse makespan $C_{\max}$, while the two rewrite fractions are of the same order; the advantage of bounded repair is a much shorter runtime (milliseconds versus seconds). Runtime and rewrite fraction must be explained separately, and Section 6.9 tests each of them. The short runtime comes from planning one path for each transport leg in the impact set without restarting a population-based search, not from the impact set itself. Whether the impact set reduces rewrites cannot be inferred from the comparison with global rescheduling above (the two fractions are already close); it requires a separate baseline that uses the same rerouting procedure and differs only in whether the rewritable set is the impact set or all unfinished occupancies. On all 12 instance–seed cells measured, global rescheduling attains a smaller $C_{\max}$ under every search time limit. When the scope is delineated on the operation-precedence graph, a corridor blockage triggers no local rescheduling because no affected operation is found, and the original schedule remains infeasible; triggering global rescheduling instead restores feasibility, but at a runtime of seconds. This paper therefore focuses on identifying the invalidated corridor occupancies, and on the corresponding local recovery, when feasibility must be restored immediately.

1.4 Contributions

The contributions of this paper are as follows; they are referred to as C1, C2, and C3.

C1. Identifying a class of invalidation that the operation-precedence graph cannot capture. When a schedule captures processing order with operation precedence and records corridor occupancies with time windows, the standard rescheduling scope is still delineated by the affected operations. Disturbances such as corridor blockage and corridor slowdown change no operation assignment, so the scope delineated by this method is empty, and the method concludes that no rescheduling is needed; yet the corridor occupancies that overlap the disturbed interval are already invalid, the original schedule is infeasible, and the resulting waits propagate to downstream vehicles through yielding relations. The root cause is that the operation-precedence graph contains no yielding relations among occupancies.

C2. Defining the rescheduling scope as the reservation impact set. Starting from the corridor occupancies that overlap the disturbance window, the scope follows yielding, the next occupancy of the same vehicle, and

the occupancies of the next operation of the same job and on the same machine, and includes every occupancy that must, directly or indirectly, be discarded and rewritten; this delineation method is called the spatiotemporal reservation closure (STRC). Once the dependencies are fixed, membership of an occupancy in the impact set is uniquely determined, and no occupancy in the set has a dependency edge leading outside it, so "what to rewrite" corresponds to a verifiable set rather than a manually chosen search radius and does not depend on how the rerouting algorithm is implemented. The evidence for *the impact set as a scope object* is as follows: under corridor blockage the scope on the operation-precedence graph is empty while the original schedule is infeasible (50/50, Section 6.2); the reservation impact set is closed under outgoing edges (50/50, Section 6.3); under Class A disturbances the reservation impact set contains every occupancy already included by the operation-precedence graph (Section 6.8); and the edge-set difference probe finds leakage on 0/50 pairs. *The net benefit of this delineation* is given by the comparison with RA (Section 6.9), namely the reduction in rewrites relative to rewriting all unfinished occupancies.

C3. Providing a local recovery matched to this delineation and separating the sources of each result. The original vehicles are rerouted inside the impact set while occupancies outside it are kept; if the schedule remains infeasible, the rewritable scope is enlarged. The repair steps follow existing practice; this contribution lies in using controlled comparisons to separate the sources of each result. Comparisons under the same corridor network, time-window routing, and conflict-free validation show the following. Under corridor blockage, scoping on the operation-precedence graph restores feasibility on 0/50 pairs and scoping on the reservation impact set on 50/50 pairs, so feasibility is determined by the scope object rather than by the quality of the rerouting procedure. Bounded repair restores feasibility within milliseconds, but its makespan $C_{\max}$ is worse than that of global rescheduling, and no crossover appears as the budget is relaxed (12 instance–seed cells, no crossing point within the measured budget range). Compared with rewriting all unfinished occupancies, runtime and $C_{\max}$ are almost unchanged, and only the reduction in rewrites can be attributed to the impact set (Section 6.9). Across four disturbance types and two scope objects, the scope on the operation-precedence graph is empty only under corridor blockage and corridor slowdown, whereas the reservation impact set is always nonempty and, under robotic-arm and vehicle failures, contains the scope on the operation-precedence graph.

In the same shop and on the same reservation table, integrated scheduling before execution uses the reservation table to build a conflict-free plan [9, 17, 20]; this paper uses the reservation table at execution time to locate the invalidated scope. The two act at different stages.

1.5 Organization

Section 2 reviews related work in four strands: how job shops delineate the rescheduling scope by operations or on the time axis, how railway scheduling represents the order of occupancies on exclusive resources, how conflict-free multi-vehicle routing performs local replanning, and where flexible job shops with transportation draw the boundary; it then positions this paper. Section 3 presents the corridor-reservation setting and the execution-time recovery problem: the shop, the corridor reservation table and the operation-precedence graph, the two classes of disturbances, and the conditions that recovery must satisfy. Section 4 defines the spatiotemporal reservation closure and its result, the reservation impact set, and establishes its dependency edges and properties such as closure under outgoing edges (Figures 2 and 3). Section 5 describes how the original vehicles are rerouted inside the impact set while occupancies outside it are kept, how the rewritable scope is enlarged when the schedule remains infeasible, and how this compares

with global rescheduling (Figure 4). Section 6 reports feasibility, runtime, the rewrite fraction of occupancies, and makespan, and uses a case Gantt chart to show, on the operation time axis, the pattern of changes behind the statistical results. Section 7 concludes and discusses limitations and future work.

2 Related work

A basic question in dynamic scheduling is on which object the rescheduling scope is delineated after a disturbance, that is, what is changed and what is kept as planned. The literature delineates this scope on different objects: operations or the time axis, the occupancy order on exclusive resources, the affected vehicles, and the set of operations in the flexible job shop with transportation. We review, for each of these four classes, what has been done and what remains uncovered, and summarize them in Table 2.

2.1 Local rescheduling in job shops

For operation-level events such as machine breakdowns and rush-order insertions, delineating the rescheduling scope by the affected operations is still common in dynamic scheduling with AGVs. Zhang et al. [24] handle machine breakdowns and rush orders, taking as the recovery scope the operations not yet processed after the breakdown. Tang et al. [19] classify operations and delivery tasks, according to the impact of the disturbance, as retained, continued, or reconstructed. They compare full rescheduling, partial rescheduling, and right shift side by side, noting that right shift preserves stability at the cost of optimization room. The original sentences underlying the paraphrases used for positioning in this and the next subsection are listed in Table 1. This practice dates back to affected-operation rescheduling (AOR) [1, 16], which reschedules only the operations directly or indirectly affected by the disturbance. Delineating the scope along the time axis and enlarging it after failure have also been studied, including match-up [2] and Local Rescheduling with incrementally enlarged time windows [8]. Right shift, in contrast, does not determine which operations are affected; it postpones the entire unfinished part. Rolling-horizon and decomposition methods trade off scale against quality [23, 26]; see [14, 21] for surveys.

These works show that delineating a scope and then rescheduling locally is not a new problem, provided that the disturbance acts on operations in the first place. Once a disturbance only makes a corridor impassable without changing any assignment, the scope delineated by this approach can be empty, even though the original schedule is already infeasible.

2.2 Occupancy order on exclusive resources

Modeling "who occupies the same exclusive resource first" as a graph has been studied systematically in railway scheduling. The alternative graph of Mascis and Pacciarelli [12] generalizes the disjunctive graph: vertices are occupations of block sections by operations, fixed arcs encode route precedence, and paired alternative arcs encode which train passes the section first. D'Ariano et al. [4] use this model for real-time train rescheduling, recomputing a conflict-free timetable after a disturbance while staying as close as possible to the original plan. Yielding relations on the same corridor express the same kind of precedence as alternative arcs.

The difference from this paper lies in how the precedence is used. For each conflict, the alternative graph provides two mutually exclusive arcs (A passes first, or B passes first), and scheduling amounts to selecting one of them. The timetable is feasible only if the selection leaves no positive-weight cycle, and the objective is optimized as far as possible; occupancy order is therefore a decision variable. In this paper the plan has already been scheduled, and who passes which corridor first is already fixed. We do not choose the order again; we only use these established yielding and

succession relations to bring the original corridor occupancies invalidated by the disturbance into the rescheduling scope. Moreover, the railway model treats block sections as machines, so operations and occupancies live on the same graph, and an unavailable section is itself an operation-level event. Hence the problem of "a scope delineated on the operation-precedence graph missing a corridor blockage" does not arise there. In the flexible job shop, processing and corridor occupancy are two separate sets of objects, and existing AOR scoping does not exploit the occupancy order.

2.3 Local replanning in conflict-free multi-vehicle routing

In multi-agent path finding (MAPF), replanning only the affected agents while treating the others as dynamic obstacles is standard practice. For online MAPF, Švancara et al. [18] propose Replan All, Replan Single, and Replan Affected, and determine the affected set by online independence detection. Malka et al. [11] likewise replan only the vehicles affected by observed changes when the map is inaccurate. Conflict-free path planning treats spatiotemporal occupancy as a first-class constraint, but start and goal locations are usually given externally [25].

These works have already made "change only the affected, keep the rest as planned" a standard mechanism, which we adopt. They differ from our setting in that they have only vehicle start and goal locations and no job–operation–machine assignment. Hence the situation in which "the rescheduling scope is delineated on the operation-precedence graph and a corridor blockage is missed for lack of an affected operation" cannot arise. As for how the affected set is obtained, MAPF finds the affected vehicles by independence detection; the result is the output of a detection algorithm and varies with its implementation and stopping condition. The reservation impact set proposed here differs: once the dependency edges are given, whether an occupancy belongs to the set depends only on whether it is reachable from the seeds along these edges, independently of the rerouting algorithm used afterwards (Section 4.2).

2.4 Flexible job shop scheduling with transportation

Flexible job shop scheduling with transport vehicles has been systematically surveyed [22]. Recent combinatorial optimization work still commonly takes travel times as a constant matrix indexed by location pairs, and builds constraint programming, metaheuristics, and knowledge-guided search on top of it [5, 13, 15]. Under this convention vehicles do not compete for the same road segment, so a corridor blockage has no counterpart in the model and cannot be tested.

Once corridor occupancies are recorded as time windows, both processing order and corridor occupancy enter the model. Congestion-aware dispatching can select vehicles using proxies such as local density, without changing machine assignments [7]. Work that jointly models flexible assignment and conflict-free transportation is relatively scarce. Van Os and Basten [20] formalize flexible job shop scheduling with conflict-free transportation constraints and, with a decomposition method, obtain provably optimal solutions on most benchmark instances; for a few, optimality cannot be guaranteed because of the time limit. Two-stage schemes first fix the production schedule and then resolve vehicle conflicts [17]. There is also a framework with a learning-based scheduling layer and a conflict-resolving routing layer, connected by limited feedback [9]. These works, however, address how to build a feasible, high-quality conflict-free schedule before execution starts.

Among the works cited in this section, a corridor reservation table and a response to execution-time disturbances never appear together: works with a reservation table exclude execution-time failures, and works that handle execution-time disturbances have no reservation table. For example (original sentences in Table 1), Zhang et al. [24] state in their modeling assumptions that collisions between AGVs are ignored. Sun et al. [17] resolve vehicle conflicts with a time-window Dijkstra algorithm, yet assume that vehicle charging and failures are not considered and that vehicles are always available; they list equipment maintenance and breakdowns as future research.

Table 1. The five paraphrases on which we base the gap, with the cited original text (English sentences quoted verbatim).

Our paraphrase	Cited original text	Location in source
Zhang et al. [24] take as the recovery scope the operations not yet processed after the breakdown	*"Rearrange the unprocessed operations of current scheme considering the limit of machine breakdown"*	Rescheduling procedure
Zhang et al. [24] state in their modeling assumptions that collisions between AGVs are ignored	*"The collision of AGVs are ignored."*	Modeling assumption (7)
Tang et al. [19] classify operations and delivery tasks by disturbance impact as retained, continued, or reconstructed	*"an event-driven partial rescheduling strategy is proposed, in which the disrupted operations and delivery tasks are classified into three categories: retained, continued, and reconstructed"*	Abstract
Van Os and Basten [20] obtain provable optimality on most benchmark instances; for a few, optimality cannot be guaranteed because of the time limit	*"For most benchmark instances, our LBBD-based approach finds optimal makespan values. Because of time boxing, only in a few cases, the optimality of the found solution cannot be guaranteed."*	Abstract
Sun et al. [17] assume vehicles are always available, ignore failures and charging, and list equipment maintenance and breakdowns as future research	*"It does not consider the charging problem and fault problem of the AGVs, and AGVs are always available"*; *"Breakdowns and charging problems are not taken into account."*; *"Future research will investigate the impact of equipment maintenance and breakdowns on production scheduling"*	Assumption list; future work

The gap therefore arises because two conditions have not been met together. Operation-level scoping is often valid precisely because the model contains no corridor reservations, while settings with a reservation table exclude execution-time disturbances. When both are present, the only directly applicable practice is scoping by affected operations, which yields an empty scope for a corridor blockage that does not change any operation assignment (measured in Section 6.2: 50/50). This is exactly the situation addressed in this paper.

2.5 Positioning of this paper

In summary, in the flexible job shop with conflict-free AGV routing, a schedule uses operation precedence to describe processing order and time windows to record corridor occupancy. This gives rise to the situation corresponding to the gap identified above: a disturbance such as a corridor blockage does not make any operation inexecutable, and the original assignment remains valid, yet the corridor occupancies overlapping the disturbed interval are invalidated, so the original schedule is infeasible. The baseline that scopes on the operation-precedence graph later in the paper reproduces the practice of Section 2.1. We propose moving the rescheduling scope onto corridor occupancies and their yielding and succession relations, which yields a reservation impact set uniquely determined by these relations.

3 The corridor-reservation setting and the execution-time recovery problem

3.1 System setting and notation

The problem setting follows the flexible job shop with conflict-free transportation constraints [20]. This subsection defines, once and for all, the objects used later. A feasible schedule $\mathcal{S}^0$ is taken as a given input: how it was found at $t = 0$ does not affect how we later delineate the occupancies that must be rewritten.

3.1.1 Shop floor and four groups of decisions. The shop routing graph $G = (V, E)$ is strongly connected. Each machine m performs processing through a robotic arm located at a fixed node $\mathrm{loc}(m)$; a robotic-arm failure below means the failure of that machine. $v_0 \in V$ is the load/unload station. There are n jobs, a machine set M, and a homogeneous fleet

Table 2. Related work classified by *scope object*: the kind of object on which each class defines "what must be rewritten" after a disturbance. "Scope delineated on" is the object to which the rescheduling scope is attached; "Operation-precedence graph?" also indicates whether that graph coincides with the occupancy graph. The bold entry in the last row marks the class to which this paper belongs.

Class	Representative work	Scope delineated on	Operation-precedence graph?	Relation to this paper
Local rescheduling in job shops	AOR [1, 16]; match-up [2]; LRS [8]; dynamic scheduling with AGVs [19, 24]	Operations / time windows	Yes	Operation-level scoping and right shift are reproduced as baselines later
Occupancy order on exclusive resources	alternative graph [12]; real-time railway rescheduling [4]	Block-section occupancies	Yes, on the same graph as occupancies	Same kind as yielding relations; railways use it to choose the order of trains, whereas we use the established order to delineate the occupancies that must be rewritten
Conflict-free multi-vehicle routing	Online MAPF [11, 18]; LMAPF [25]	Affected vehicles	No	Rerouting inside the set and keeping the rest as planned is already standard
Flexible job shop with transportation	constant matrix [5, 13, 15, 22]; integrated scheduling [9, 17, 20]	Set of operations	Yes; reservation table and execution-time disturbances never appear together	**Position of this paper**

of unit-load vehicles $K = \{1, \dots, N_A\}$. Job j has a fixed chain of operations $O_{j1} \to \cdots \to O_{jn_j}$; operation O_{ji} has a candidate machine set $\Omega_{ji} \subseteq M$ and processing time t^P_{jim} on machine m. A complete schedule fixes four groups of decisions simultaneously: (i) the machine assignment x of each operation; (ii) the processing sequence π of operations on each machine; (iii) the vehicle assignment y of each transport; (iv) a conflict-free path on G for each vehicle move. We write

$$\mathcal{S} = (x, \pi, y, R),$$

where the reservation table R is the explicit record of group (iv), defined below.

Transport tasks are *derived* from the assignment rather than given as input: a transport arises only when two consecutive operations are at different nodes. Each transport is split into two legs, an empty leg for pickup and a loaded leg for delivery, each with its own task label u (for example, the empty leg and the loaded leg of O_{ji} are two separate records). Loading and unloading times are zero: the loaded leg departs at the later of the empty leg's arrival and the job's ready time, and the delivery time is simultaneously the time at which the machine can start and the vehicle can accept a new task. Which occupancies are rewritable, which are kept as planned, and how rerouting is done are all decided at the granularity of legs, not whole operations.

3.1.2 Corridor reservation and conflict freedom. Each corridor $e \in E$ is an exclusive resource with known traversal time τ_e. Both directions share the same occupancy, so at most one vehicle occupies e at any time, which rules out both head-on conflicts and same-direction overtaking. Occupancies are half-open intervals: a second vehicle may enter exactly at the right endpoint of the interval. Nodes have ample parking capacity, so waiting takes place at nodes and is

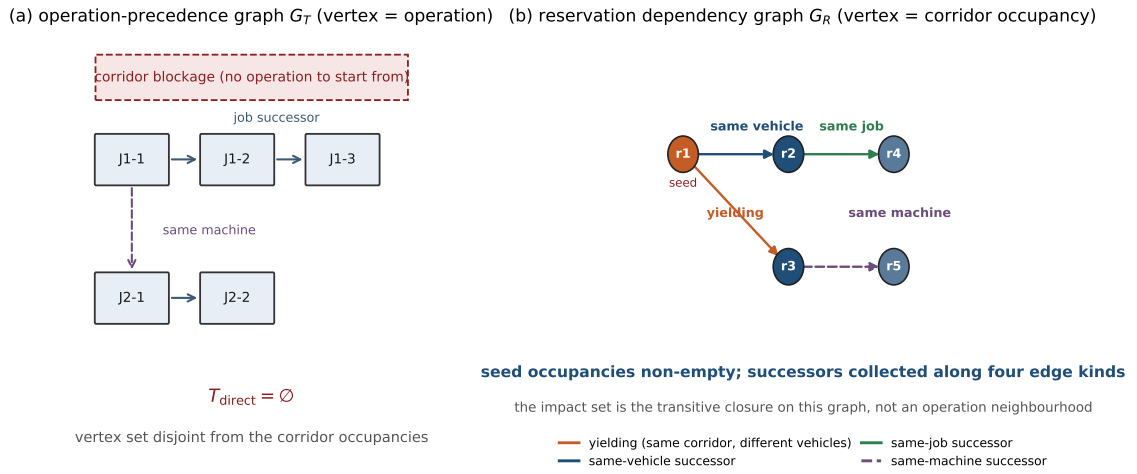


Fig. 2. The vertex sets of the two graphs are disjoint (schematic construction). (a) Operation-precedence graph G_T: vertices are operations, and a corridor blockage finds no seed operation. (b) Reservation dependency graph G_R: vertices are corridor occupancies; starting from the occupancies overlapping the blockage window, subsequent occupancies are included along the four kinds of dependency edges (yielding, same vehicle, same job, same machine; Definition 4).

always allowed. A reservation record is

$$r = (e,\ [t^s, t^e),\ k,\ u) \in R,$$

meaning that vehicle k occupies corridor e during $[t^s, t^e)$ for leg u. Any two distinct occupancies of the same corridor must satisfy $[t^s, t^e) \cap [t^{s\prime}, t^{e\prime}) = \emptyset$. Paths are written only into time windows that R reports as free, so conflict freedom holds by construction.

Time-window routing is the primitive shared by our rerouting and by all baseline methods: given the origin and destination, the earliest departure time, and the current R, it returns a path disjoint from existing occupancies together with the entry time of each corridor. We implement it as a time-window Dijkstra algorithm [17]. We do not modify this primitive; we change only *which* transport legs are sent to the router. Unlike van Os, who reserves nodes, and Lyu, who additionally imposes node capacities [10, 20], we reserve edges; Section 7.1 discusses the resulting restrictions on instance scope.

3.1.3 Operation-precedence graph and reservation dependency graph. The operation-precedence graph $G_T = (T_T, E_T)$ (also called the task graph in the literature; the subscript T stands for task) has operations as vertices and job precedence and same-machine precedence as edges; it does *not* contain the yielding relations between corridor occupancies. The reservation dependency graph $G_R = (R, E_R)$ has the occupancies in R as vertices, and its edges are the dependencies induced by yielding, same vehicle, same job, and same machine (Definition 4). The vertex sets of the two graphs are disjoint: one represents processing, the other traversal. Hence there is a class of disturbances that changes only corridor time windows and no operation parameter. Scoping on G_T finds no seed operation, whereas scoping on G_R yields a nonempty seed set, and the original schedule is already infeasible. Figure 2 contrasts the two graphs, Figure 1 shows the consequences under a corridor blockage, and Table 3 summarizes the notation used in this section and later.

Table 3. Frequently used notation

Symbol	Meaning
$G = (V, E)$, τ_e	Corridor network; traversal time of corridor e
v_0, $\mathrm{loc}(m)$	Load/unload station; node of robotic arm m
x, π, y	Machine assignment, processing sequence, vehicle assignment
R, $r = (e, [t^s, t^e), k, u)$	Reservation table; a half-open occupancy (vehicle k, leg label u)
G_T / G_R	Operation-precedence graph / reservation dependency graph
$\mathcal{S}^0$, $\mathcal{S}'$	Schedule before the disturbance, schedule after recovery
$\mathcal{D}$, t_{now}	Disturbance event; occurrence (decision) time
T_{direct}, T_{impact}	Directly disturbed operations on the operation-precedence graph / θ-step scope
$\mathrm{Src}(\mathcal{D})$, U	Seed occupancies; set of rewritable occupancies
$\mathrm{Cl}(\cdot)$	Reservation impact set (obtained by STRC)
φ	Fraction of disturbed future operations (magnitude sweep)

3.2 Execution-time assumptions

The execution-time conventions and physical setting are as follows. A1–A4 are shared by all compared methods, so that differences in results can be attributed to how the rescheduling scope is delineated rather than to model differences. A5 constrains only our method; the global-rescheduling baseline is allowed a wider set of actions (see the last sentence of A5):

A1. Feasible initial schedule. $\mathcal{S}^0$ passes the conflict-freedom check before the disturbance, and the reservation table is consistent with the operation timetable.

A2. No backtracking on finished occupancies. Only unfinished occupancies with $t^e > t_{\mathrm{now}}$ and unfinished operations are rewritten; the fields of occupancies with $t^e \le t_{\mathrm{now}}$ are not rewritable.

A3. Single, fully observed disturbance. The disturbance is fully observed at t_{now} and does not overlap with other disturbances; disturbance streams, prediction errors, and rolling responses are beyond the scope of this paper.

A4. Exclusive corridors, ample buffers, homogeneous unit-load fleet. Corridors remain exclusive half-open resources, and a blockage is injected into the routing layer as an additional occupancy (or no-entry window). Finite buffers, charging, acceleration and deceleration, and loading/unloading durations are not modeled; all vehicles start empty at v_0 at the beginning of the plan.

A5. Permitted recovery actions. For Class B disturbances, the original machine assignment x and processing sequence π are kept and only rerouting is performed; upon failure, the set of rewritable occupancies is escalated. Class A failures additionally allow the following: a vehicle failure removes the failed vehicle from the fleet and reassigns its tasks to another vehicle; a robotic-arm failure reassigns unfinished operations to other feasible machines. The processing sequence π is never changed, and the search is not restarted. The global-rescheduling baseline in the experiments may change x and π (Section 5.3).

3.3 A dichotomy of disturbances

Definition 1 (Class A/B disturbances). *A disturbance is of **Class A** if it directly invalidates the assignment or* executability *of some operations (robotic-arm failure, significant processing-time deviation, rush-order insertion, vehicle failure, etc.). It is of **Class B** if it only makes some corridor time windows unavailable without making any operation inexecutable (corridor blockage, corridor slowdown, etc.).*

Remark 1 (Why vehicle failure is Class A). *A vehicle failure does not change the machine assignment of any operation, so at first sight it seems to belong with corridor blockages. However, the criterion in Definition 1 is* executability, *not assignment: after a vehicle fails, the operations it transports cannot start under the original plan. Through the mapping "vehicle→operations it transports", the operation-precedence graph therefore still contains seed operations. This classification places vehicle failure on the side where operation-level scoping gives a nonempty scope. These seed operations are nonempty only if the rescheduling method maintains a vehicle–task mapping; if the method tracks only operations and machines (as common AOR implementations do), a vehicle failure also yields an empty scope. Section 6.8 reports Class A results under the convention that this mapping is maintained.*

Definition 2 (Direct set and affected set on the operation-precedence graph). *For a disturbance $\mathcal{D}$, let $T_{\text{direct}}(\mathcal{D})$ be the set of operations that cannot be executed as planned because of a robotic-arm failure, a vehicle failure, or operation invalidation. For Class B corridor disturbances, we set $T_{\text{direct}} = \varnothing$ by convention. Starting from T_{direct} on G_T, the operations reached along job-successor and same-machine-successor edges within depth at most θ form $T_{\text{impact}}(\mathcal{D}, \theta)$.*

Definition 3 (Seed occupancies). *The seed set $\mathrm{Src}(\mathcal{D}) \subseteq R$ is given by disturbance type and always includes only occupancies with $r.t^e > t_{\text{now}}$.*

(a) ***Corridor blockage/slowdown*** *$\mathcal{D} = (e, [t_a, t_b))$: $\{\, r : r.e = e,\ [r.t^s, r.t^e) \cap [t_a, t_b) \neq \varnothing \,\}$. For multiple short blockages, the union of the seed occupancies of all windows is taken. Blockage and slowdown share this seed rule (the original schedule was written with the unscaled τ, so every occupancy overlapping the window is invalidated); hence they yield the same reservation impact set when delineating the occupancies to be rewritten. Section 6.8 uses this to explain why their scoping results are not counted as two independent validations. They differ in rerouting: a blockage is written as a no-entry window, whereas a slowdown only scales, by a factor, the traversal progress in the part overlapping the window.*

(b) ***Vehicle failure*** *$\mathcal{D} = (k)$: all occupancies of that vehicle after t_{now}, $\{\, r : r.k = k \,\}$.*

(c) ***Robotic-arm failure*** *$\mathcal{D} = (m, F)$, where F is the set of unfinished operations on that machine: for each job, take the smallest operation index $i_{\min}(j)$ in F; the seed occupancies are all unfinished occupancies of each job from that operation onward (inclusive). We take the job chain forward rather than only the operation itself because its inbound transport may have finished before t_{now}; taking only the operation itself would make the reservation impact set shorter than the affected set on the operation-precedence graph, and the comparison would be unfair.*

Proposition 1 (The operation-precedence graph yields an empty scope for Class B). *Let $\mathcal{D}$ be a corridor blockage. Under the Class B convention of Definition 2, $T_{\text{direct}}(\mathcal{D}) = \varnothing$, and hence $T_{\text{impact}}(\mathcal{D}, \theta) = \varnothing$ for every finite $\theta \geq 0$. If, in addition, $\mathrm{Src}(\mathcal{D}) \neq \varnothing$, then $\mathcal{S}^0$ is infeasible under $\mathcal{D}$.*

Proof. A corridor blockage only makes corridor time windows unavailable without making any operation inexecutable, so it is of Class B by Definition 1. This step relies on two premises: the blockage window $[t_a, t_b)$ is finite, and by Assumption A4 buffers are ample, so vehicles can traverse outside the window. Hence, even if the network is disconnected during the blockage, no operation becomes inexecutable; operations are only delayed. If the blockage were indefinite, or buffers were insufficient to support waiting, the disturbance would be of Class A by Definition 1 and would fall outside the scope of this proposition. Being of Class B, it has $T_{\text{direct}} = \varnothing$ by the Class B convention of Definition 2. The starting set of the θ-step expansion is empty, so $T_{\text{impact}} = \varnothing$. If some $r \in \mathrm{Src}(\mathcal{D})$ exists, the occupancy interval of r intersects the no-entry window. Interpreting $\mathcal{D}$ as a mandatory occupancy of that corridor, r and the no-entry window overlap in time on the same exclusive resource, which violates the half-open time-window conflict-freedom condition (corridor exclusivity in Assumption A4). Hence $\mathcal{S}^0$ is infeasible. □

Proposition 1 is a counterexample at the problem level. Its first part follows directly from the Class B convention of Definition 2 (by definition, the operation-precedence graph does not contain the yielding relations between corridor occupancies and thus cannot represent a corridor blockage). The nontrivial part is the second: when the seed set is nonempty, the original schedule is already infeasible. Hence "an empty scope on the operation-precedence graph ⇒ no rescheduling needed" does not hold in a setting where corridors are reserved by time windows. The counterexample does not depend on any repair algorithm; Section 6 reports its extent on the instances.

3.4 The recovery problem

Problem 1 (Bounded rescheduling with the scope delineated on the reservation table). *Given a feasible schedule $\mathcal{S}^0$, a disturbance $\mathcal{D}$, and a time t_{now}, find $\mathcal{S}'$ such that (i) $\mathcal{S}'$ passes the conflict-freedom check under $\mathcal{D}$; (ii) occupancies outside the rewrite scope keep their corridors, time intervals, and vehicles as close to $\mathcal{S}^0$ as possible; (iii) the runtime is subject to a budget. The secondary objective is the makespan $C_{\max}(\mathcal{S}')$.*

Bounded rescheduling proceeds in two steps: first delineate the set of rewritable occupancies $U \subseteq R$; then reroute only the transports in U, keeping occupancies outside the set as planned; we call the second step *bounded repair* (Section 5). We obtain U as Cl(Src); the baselines instead derive a reservation subset from the affected set on the operation-precedence graph.

4 The spatiotemporal reservation closure

This section produces only the rewritable occupancy set U. It takes as input the reservation table R and the seed set $\text{Src}(\mathcal{D})$ (Definition 3), builds a dependency graph over occupancies, takes its transitive closure, and outputs $U = \text{Cl}(\text{Src})$. How vehicles are rerouted on U is the subject of Section 5.

4.1 Reservation dependency edges

Definition 4 (Reservation dependency graph). *The vertices are the corridor occupancies in R. A directed edge $r \to r'$ means "invalidating r may force r' to be rerouted". The edges are of four kinds:*

(1) ***Yield edges:*** *two occupancies of the same corridor by different vehicles whose time intervals overlap* or abut as half-open intervals *($r.t^e = r'.t^s$); the edge points from the earlier entrant to the later one. Abutting is legal under exclusive semantics, but if the former is delayed, the latter must give way.*

(2) ***Same-vehicle successors:*** *two occupancies of the same vehicle that are adjacent in time order.*

(3) ***Same-job successors:*** *an edge from the occupancies of operation i of a job to those of operation $i+1$ of the same job.*

(4) ***Same-machine successors:*** *edges between the occupancies induced by two operations that are adjacent on the timeline of the same machine.*

This yields $G_R = (R, E_R)$. Figure 2(b) illustrates the four kinds of edges. Same-job successor edges connect the occupancies induced by consecutive operations, and same-machine successor edges connect the occupancies induced by two operations adjacent on the timeline of one machine; both have legs as endpoints, consistent with the granularity of Section 3.1.1. The four kinds of edges answer a single question: after r is rewritten, might r' have to be rewritten as well? The edge set is a modeling choice; its completeness is tested by the difference probe of Section 6.4.

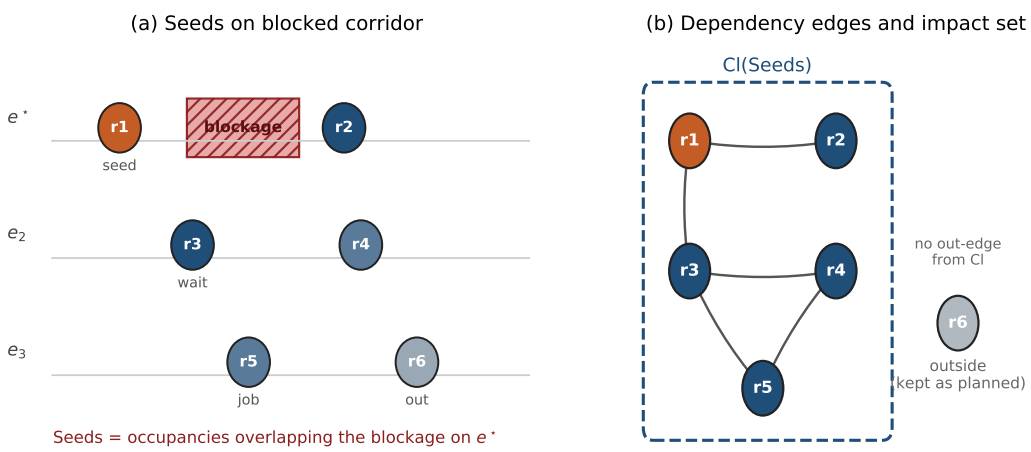


Fig. 3. Construction of the reservation impact set. (a) Occupancies that overlap the corridor blockage window serve as seeds; (b) taking the transitive closure along the four kinds of dependency edges (yielding, plus the same-vehicle, same-job, and same-machine succession relations) yields Cl(Src); occupancies outside the set are kept as planned.

Definition 5 (STRC). *Given a seed set $S \subseteq R$, a horizon H, and a time t_{now}, denote the occupancies that are unfinished at the decision time and start before the horizon by*

$$R^{\circ} = \left\{ r \in R : \; r.t^{e} > t_{\text{now}}, \; r.t^{s} < H \right\},$$

and let G_R° be the subgraph of G_R induced by R°. The result of the spatiotemporal reservation closure, namely the reservation impact set, is defined as

$$\text{Cl}(S) = \left\{ v \in R^{\circ} : \; \exists s \in S \cap R^{\circ} \text{ such that } s \rightarrow^{*} v \text{ in } G_R^{\circ} \right\},$$

that is, all unfinished occupancies reachable along outgoing edges from the unfinished seeds (the seeds themselves included). All experiments in this paper use $H = C_{\max}(\mathcal{S}^0) + 1$. Then $r.t^s < H$ holds for every occupancy in $\mathcal{S}^0$ and the horizon filters nothing; we keep H only so that the definition also applies to a rolling horizon. If G_R° is stored as adjacency lists (with edge set E_R°), the set can be computed with a queue in $O(|R^{\circ}| + |E_R^{\circ}|)$ time. We did not time this step separately; delineating the scope is only one component of the millisecond-range results of Section 6.9.

Figure 3 gives an illustration: (a) the blockage window and the occupancies taken as seeds; (b) the dependency edges and the boundary of the reservation impact set. No occupancy outside the set is an outgoing-edge successor of an occupancy inside it (Proposition 2).

The baseline that scopes on the operation-precedence graph maps T_{impact} to rewritable occupancies: all occupancies whose task label belongs to the affected set and whose $t^e > t_{\text{now}}$. By Proposition 1, this set is empty under Class B disturbances. Scoping on the reservation impact set takes $U = \text{Cl}(\text{Src})$. In the experimental tables the two are denoted R1 and R2, respectively.

4.2 Containment

Proposition 2 (Structural closure). *For any $S \subseteq R$, G_R° contains no edge $u \to v$ with $u \in \mathrm{Cl}(S)$ and $v \in R^\circ \setminus \mathrm{Cl}(S)$. Consequently, no unfinished occupancy outside the impact set is reachable along outgoing edges from* $\mathrm{Cl}(S)$.

Proof. Suppose such an edge $u \to v$ exists. Since $u \in \mathrm{Cl}(S)$, there is an unfinished seed s with $s \to^* u$. Appending $u \to v$ gives $s \to^* v$, so $v \in \mathrm{Cl}(S)$ by Definition 5, a contradiction. □

Proposition 2 shows that "local" corresponds to a graph-theoretically closed object: no occupancy outside the impact set is a direct successor of any occupancy inside it. Once the edge set is fixed, closure is a corollary of the definition; its physical completeness depends on how the edge set is chosen.

Remark 2 (The reservation impact set and the minimal rewritable set). *Structural closure states only that the impact set is closed under the modeled dependency edges: no dependency edge leads from inside the set to outside it. It neither guarantees that the impact set suffices to restore feasibility nor shows that the impact set is not too large. For the latter there are three reasons.*

First, ***the minimal rewritable set depends on the repair procedure used, whereas the reservation impact set does not****. To call a set U minimal, one must specify "under which repair capability U suffices to restore feasibility". The minimal U when reassignment and resequencing are allowed differs from the minimal U when only rerouting is allowed. The reservation impact set is uniquely determined by G_R and independent of the repair procedure. Hence, by definition, it is not tied to any particular heuristic, and it cannot simultaneously be a procedure-dependent minimal set.*

Second, ***the four kinds of edges admit occupancies that need not be rewritten, while the known case of missed occupancies, abutting, has been folded into the yield edges****. Yield edges are built on "same corridor, different vehicles, overlapping or abutting": any overlap creates an edge, without checking whether the latter* truly *can proceed only by waiting for the former to pass. If a bypass exists there, the latter can simply detour and need not enter the rewritable set. Likewise, same-vehicle successor edges may pull in chain tails that the delay cannot reach. Both effects make the impact set larger than necessary. In the opposite direction, half-open intervals that meet end to start (abutting) are legal under exclusive semantics, but if the former is delayed the latter must give way; abutting is therefore also connected by yield edges. The difference probe finds 0 leaks over* 50 *pairs (0/50; Section 7.1). This test targets the known abutting omission and does not prove the edge set complete.*

Third, ***empirically, the impact set is not small****. Section 6.8 reports that under corridor blockage the reservation impact set covers on average about 92% of the occupancies still unfinished after t_{now}. Relative to the trivial upper bound of "rewriting every occupancy unfinished at the decision time", the impact set leaves out less than one tenth. The savings of bounded repair come mainly from* not rewriting finished occupancies *and* not restarting the search, *not from a sharp reduction of the future occupancies to be rewritten; "local" and "bounded" do not mean that the changes are small.*

"Less than one tenth" is a ratio of the sizes *of rewritable sets, not a difference in rewrite fraction. Section 6.9 measures the trivial upper bound above as a separate baseline (RA) and finds that this very difference of less than one tenth lowers the rewrite fraction from 0.604 to 0.540 (per cell 31/18/1, $p = 8.8 \times 10^{-7}$), while runtime and makespan remain nearly unchanged.*

What this paper provides, therefore, is a rewritable set that is uniquely determined by structure, reproducible, and closed under the dependency edges, and that sufficed to restore feasibility in all our experiments. Approximating the minimal rewritable set is a separate problem, left for future work (Section 7.2).

Proposition 3 (A sufficient condition for unchanged fields outside the set). *Let the rewritable occupancy set be $U = \mathrm{Cl}(S)$, and let level-1 rerouting be executed as in Section 5.1: (i) the no-entry windows have been written into the routing layer; (ii) for every $r \in R^{\circ} \setminus U$, its original fields $(e, [t^s, t^e), k)$ are pre-committed to the new reservation table and the pre-commit succeeds; (iii) the router is invoked to generate new occupancies only for transports in U; (iv) the final schedule passes the conflict-free check. Then, in the recovered reservation table R', the corridor, interval, and vehicle of every $r \in R^{\circ} \setminus U$ agree with $\mathcal{S}^0$. The quantifier here ranges over* occupancies*, not tasks: a transport may have some occupancies in U and others outside it, in which case only the part outside the set is protected by this proposition.*

Proof. By (ii), the occupancies outside the set are written into the new table with their original intervals before the replay begins, and they do not conflict with the no-entry windows (otherwise the pre-commit would fail). Step (iii) commits new occupancies only for tasks in U and neither deletes nor rewrites the committed intervals outside the set. Hence, for task labels outside the set, $(e, [t^s, t^e), k)$ in the new table equals the pre-committed value, and thus equals the value in $\mathcal{S}^0$. Passing the check only rules out conflicts between new and existing occupancies; it does not alter committed fields. □

Premises (i)–(iv) are execution conditions of the rerouting procedure in Section 5.1 and can be verified against the algorithm one by one. They differ in kind from model-level assumptions such as A1–A5, so we do not add them to the numbered assumptions. Beyond them, the proposition relies on A1 (the initial schedule is feasible; otherwise the original fields outside the set might conflict with one another and the pre-commit would necessarily fail), A2 (finished occupancies are not revisited), and A4 (corridors are exclusive).

Remark 3 (How premises (ii) and (iii) can each fail). *If the pre-commit fails because "an occupancy outside the set overlaps a no-entry window", then S does not cover all occupancies overlapping the blockage window, and the seed set is incomplete. If rerouting fails at a transport–operation hand-off and triggers the scope escalation of Section 5.2, the new rewritable set satisfies $U' \supsetneq U$, and Proposition 3 holds only for the occupancies that are ultimately left outside the rewritable set. Scope escalation puts feasibility ahead of minimal rewriting and is reported separately in the experiments. One further failure lies outside what the proposition controls but occurs easily in implementations: premise (iii) requires the router to generate new occupancies* only *for transports in U. If an implementation decides whether to reroute at a unit coarser than an occupancy (e.g., a whole transport or a whole operation), occupancies outside U are rewritten along with it, premise (iii) no longer holds, and the conclusion of the proposition fails. Remark 4 records an instance of this in our implementation and its fix.*

4.3 Structural predictability

The size of the reservation impact set is uniquely determined by G_R; once the schedule and network are given, there is no tunable parameter. Section 6.11 scales the instances from $8 \times 4 \times 4$ up to $20 \times 10 \times 10$; the impact-set share fluctuates between 0.48 and 0.56 and does not rise with size (this share uses the full table as denominator, with an attainable upper bound of about 0.61; relative to the unfinished occupancies R° it is 0.78–0.91). Hence the extent of "local" is measurable, but its size is not small (Remark 2).

As for *which* structure corresponds to a larger impact set, the experimental result is negative. We examine a class of indicators computed from the network alone and independent of the current schedule: the minimum number of corridors that must be cut to separate the load/unload stations from the rest of the shop, i.e., the LU minimum cut (hereafter the LU cut), together with the funnel share; we refer to this family of indicators as static cuts below. **The impact-set size**

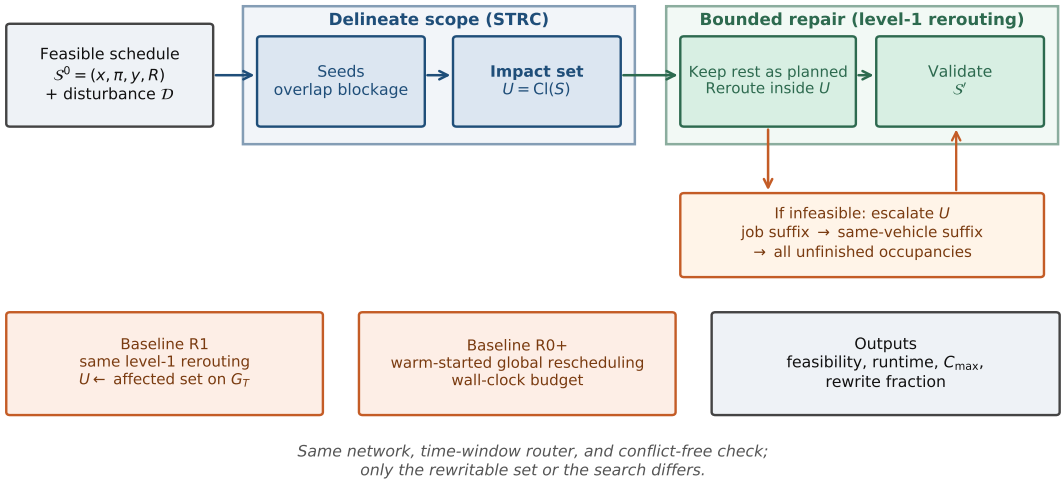


Fig. 4. Bounded repair scoped on the reservation impact set. Top row: the impact set is obtained from the seeds and vehicles are rerouted; middle row: the rewritable scope is escalated after failure; bottom row: the baseline scoping on the operation-precedence graph, the warm-started global rescheduling baseline, and the output metrics. The figure shows only the three method-level paths; three further baselines operate within the millisecond range (Section 6.9).

is not determined by static cuts. On our own layouts of the same size, under different boundary-blockage protocols, the relative size either falls within a very narrow band or shows a direction but with values heavily overlapping across random seeds. On a batch of networks not designed by us, the LU cut and the funnel share are *constant*, whereas the impact-set share changes markedly across three machine configurations on the same network (6, 7, and 8 machines). A constant indicator cannot explain a varying quantity (Section 6.6). This negative result concerns static cuts only; because few public networks are available, the general relation between network structure and impact-set size remains undetermined. The next candidate characterization should be a schedule-dependent quantity rather than a purely topological one (Section 7.2).

5 Bounded repair on the reservation impact set

Figure 4 outlines the procedure of this section. We first delineate the rewritable occupancy set U by the reservation impact set, then reroute only the transports inside the set while keeping occupancies outside it as planned; if the result is still infeasible, we escalate U and retry. The baseline that scopes on the operation-precedence graph replaces only the source of U; the global rescheduling baseline searches anew within a time limit. All methods share the same corridor network, time-window routing, and conflict-free check.

5.1 Rewritable set, kept-as-planned set, and level-1 rerouting

Given the rewritable occupancy set U, level-1 rerouting keeps the original vehicle and machine assignments and replays in the original processing order. Rerouting operates at the granularity of corridor segments (a segment is the portion of one trip that traverses a single corridor). Segments a vehicle has already traversed before t_{now} are finished occupancies

Algorithm 1 Level-1 rerouting replay ReplayReroute

Require: schedule $\mathcal{S}^0 = (x, \pi, y, R)$, disturbance $\mathcal{D}$, rewritable set U, time t_{now}
Ensure: schedule $\mathcal{S}'$ or failure

```
write D into the routing layer                ▷ blockage: no-entry window; slowdown: overlapping portion τ
take an empty table R'
for each non-rewritable occupancy: all those outside the set, and each r ∈ U with r.t^e ≤ t_now do
    pre-commit the original fields (e, [t^s, t^e), k) to R'   ▷ the second kind are finished occupancies under A2; omitting them amounts to rewriting them
    if conflict with the blockage window then
        return failure                        ▷ the seeds do not cover an occupancy overlapping the window
    end if
end for
for each r ∈ U, in the original processing order do
    plan from the node where the vehicle is at t_now ▷ traversed segments were pre-committed in the previous step and are not committed again
    invoke time-window routing on the current R' and commit the new occupancies ▷ finished occupancies are not rewritten
end for
advance machine idle times and job ready times according to π
independent check: no conflicts, no residual no-entry windows, slowdown overlap durations absorbed, fields outside the set unchanged
if the check passes then
    return S' = (x, π, y, R')
else
    return failure
end if
```

and are pre-committed unchanged; rerouting plans only from the node where the vehicle is at that time. A granularity coarser than segments would rewrite finished occupancies (Remark 4). Algorithm 1 gives this primitive; Algorithm 2 wraps it, computing the reservation impact set and escalating U on failure.

The single breadth-first pass that delineates the scope (Definition 5) accounts for only a small fraction of the runtime. One call of ReplayReroute also invokes a time-window shortest path once for each of the $|U|$ transport legs, and the measured runtime is dominated by the latter (Sections 6.4 and 6.9). This level launches no population-based search. The comparison in Section 6.4 fixes Algorithm 1 and replaces only the source of U (the operation-precedence graph or the reservation impact set), so that the difference in feasibility rate can be attributed to the scope object.

5.2 Scope escalation after failure

Escalating the rewritable scope has precedents. Local Rescheduling [8] relaxes the time window step by step until feasibility is reached; AOR adds subsequent operations along same-job and same-machine successors (Section 2.1). We escalate U along reservation dependency edges rather than along time windows. These steps put the impact set into practice: when rerouting fails, we first enlarge the set of occupancies to be rewritten instead of switching immediately to a global search. Section 6.1 explains why the boundary ablation must disable this step.

When keeping the occupancies outside the set is too restrictive, Algorithm 1 may fail checks such as the transport–operation hand-off. We then escalate U in the following order and retry the same rerouting procedure: (i) $U \leftarrow U \cup \{$the unfinished occupancies in the same-job suffixes of the operations currently in $U\}$; (ii) $U \leftarrow U \cup \{$the unfinished

occupancies in the same-vehicle suffixes of the vehicles involved}; (iii) $U \leftarrow \{\, r \in R :\ r.t^e > t_{\text{now}} \,\}$ (every occupancy unfinished at the decision time becomes rewritable). After escalation, rerouting remains a deterministic single pass. In the experiments this step restored feasibility in all cells, with runtime staying in the range of milliseconds to a little over ten milliseconds (Section 6). Runtime is not monotone in the disturbance ratio φ. One possible explanation is two opposing effects: at small ratios the initial impact set is tight and escalation is needed more often; at large ratios the initial U is already wide and one round often succeeds, but each round has more transports to reroute (results in Section 6.10).

5.3 Comparison with global rescheduling

The global rescheduling baseline (denoted R0+) searches anew from the original schedule, under the same no-entry constraints and within a time limit. Internally it encodes (x, π) as a chromosome. One *decoding* means: following the chromosome's assignments and processing order, invoke the time-window routing of Section 3.1.2 from scratch for all transports on the reservation table into which the no-entry windows have been written, obtaining a new R and $C_{\max}$. R0+ decodes repeatedly and may modify the chromosome; RD in Section 6.9 decodes only once and does not modify the chromosome. R0+ decoding does not rewrite finished occupancies: these are pre-committed, and only operations not yet started are scheduled from t_{now} on (Section 6.7). The baseline is warm-started rather than searching from scratch, so as not to discard the assignment and path information already in the original schedule. Because R0+ can reassign and resequence, it is more likely to shorten $C_{\max}$, but its runtime is set by the budget.

Bounded repair is not better than R0+ in makespan. Section 6.7 shows that R0+ already dominates at the smallest budget level; since the result of R2 does not vary with the budget and the $C_{\max}$ of R0+ is monotonically non-increasing in the budget, no crossover appears as the budget grows. The strength of bounded repair lies in runtime: when a response is required within milliseconds, it restores feasibility; when a response time of seconds is acceptable, R0+ should be used to shorten $C_{\max}$. The corridor, interval, and vehicle of occupancies outside the set are kept as planned, as guaranteed under its conditions by Proposition 3.

Comparing only with R0+ leaves one question open: within a millisecond runtime, is scoping on the impact set necessary? To answer it, we implement three approaches that do not restrict the scope by the impact set. Global right shift (RS) does not determine which occupancies are affected; it simply pushes all unfinished occupancies past the end of the blockage. Re-decoding of the original chromosome (RD) leaves the chromosome unchanged and performs a single fixed-prefix decoding. **RA** makes every occupancy still unfinished after t_{now} rewritable and then runs *exactly the same* rerouting procedure as in Section 5.1. It therefore differs from our method only in "which occupancies are rewritable" and is the only baseline that isolates the effect of the reservation impact set. All three run within milliseconds and thus serve as baselines for bounded repair *within the same runtime range*; their definitions and results are given in Section 6.9.

Algorithm 2 combines this section and the previous one, corresponding to Figure 4.

6 Experiments

6.1 Protocol

Network, routing, and instances. The lower-level network, the time-window router, and the independent checker are fixed. The extended experiment matrix uses **five instances**: the small instance `example_3x3x2`, the congested instance `congested_8x4x4`, and three `S8x4x4` layouts, `high`, `funnel`, and `mid`. The last three share the same size and flexibility

Algorithm 2 Bounded repair on the reservation impact set (level-1 rerouting + scope escalation)

Require: schedule $\mathcal{S}^0$, disturbance $\mathcal{D}$, time t_{now}
Ensure: schedule $\mathcal{S}'$ or failure
$S \leftarrow \text{Src}(\mathcal{D})$; $U \leftarrow \text{Cl}(S)$
for round = 0, 1, 2, 3 **do**
  $\mathcal{S}' \leftarrow \textsc{ReplayReroute}(\mathcal{S}^0, \mathcal{D}, U, t_{\text{now}})$
  if $\mathcal{S}'$ is feasible **then**
    return $\mathcal{S}'$
  end if
  update U by the scope-escalation rule ▷ job suffix / same vehicle / all unfinished
end for
return failure

and differ only in corridor topology, so differences in impact-set size among them can be attributed to topology rather than to problem size. The five instances come from two sources. The first two are hand-crafted layouts (the small instance allows step-by-step checking; the congested instance has a dumbbell topology). The last three are produced by our instance generator; its parameters (number of jobs × number of machines × number of AGVs, congestion level, heterogeneity h, and flexibility f) are encoded in the instance file name together with the **generation seed**, which is 42 for all three. **The generation seed is independent of the run seeds used below**: the former determines only the instance itself, whereas the latter determine the disturbance injection and the search process. Both sets of values contain 42, but no seed is shared between them. **We use ten run seeds**, namely 42, 7, 2024, 99, 123, 13, 1, 777, 31415, and 8, giving $5 \times 10 = 50$ instance–seed pairs; each pair is called a "cell" below. All cell-wise comparisons, win/loss counts, and paired tests in this paper are aligned on the key (instance, random seed): under the same key, all methods read the same initial solution, the same blockage, and the same decision time. The initial schedule is obtained by replaying the time-window router of Section 3.1.2 in processing order, and it passes the independent conflict-free check; all baseline methods share this initial solution within the same instance–seed pair. We require only that $\mathcal{S}^0$ be feasible before the disturbance.

Implementation and computing environment. All experiments are implemented in pure Python with the standard library only, without third-party numerical libraries; the lower-level network, the time-window router, and the independent checker reuse the same implementation as our static integrated work. Everything runs on a single machine in a single thread, without parallelism: Intel Xeon W-1270 (8 cores, 16 threads), 64 GB RAM, Windows 10, and the CPython 3.7.6 interpreter (64-bit). Because no third-party numerical library is used, reproduction requires no pinned dependency versions; 3.7 is the interpreter we tested, and newer versions have not been rerun one by one. All timing results were obtained under the single-machine, single-thread setting above. Different hardware would shift them uniformly, so all our runtime conclusions are stated as *ratios* or *orders of magnitude* and do not rely on absolute milliseconds.

Code and data availability. The reproduction materials consist of three parts: the upper-level bounded repair and experiment drivers; the lower-level network and time-window router; and the instances, disturbance inputs, and all cell-wise results (CSV). All three, together with the scripts that generate every table and figure, are submitted and are available during review as an **anonymous repository** (see the link attached in the submission system; the repository contains no author-identifying information). Upon acceptance, the repository will be made public and registered with a DOI in a permanent archiving service; this section will then be updated with that DOI. Every table and figure can be

traced in the repository to the script that generates it and to the corresponding CSV, and the instances listed in Table 13 are distributed with the repository.

Second batch: externalizing the layout source. All five instances above use layouts designed by us, and the three same-size layouts also belong to one dumbbell family (they differ only in the number of parallel lanes at the load/unload station exit and in the middle section, that is, only in capacity, not in geometry). To ensure that the layouts are not chosen by us, we run an additional batch of 5 instances. Three items of the network—the grid size, the placement of the load/unload station and the machines, and the missing edges—are transcribed item by item from the layout figures in Appendix A of Lyu et al. [10], and the transcription is reconciled field by field by a script against the machine-readable encoding provided with the van Os toolkit. The number of machines is therefore determined by the layout rather than chosen by us, and ranges from 4 to 8. The same ten random seeds are used, giving 50 instance–seed pairs in total. The two batches are *matched parameter by parameter*: the number of jobs, the number of AGVs, the number of operations per job, h, f, and the calibration target of the transport-to-processing time ratio are all identical, so the difference between the two batches can be attributed solely to the layout source.

That appendix gives six layouts in total (3 to 8 machines). We do not use the 3-machine layout: with $f = 0.6$ fixed and $|M| = 3$, the average number of candidate machines is $1.8 < 2$, which is incompatible with the requirement that "every operation has at least two candidate machines"; relaxing f for a single layout would introduce an extra variable.

The remaining five layouts **fall on only 2 distinct networks**: the 4-machine and 5-machine layouts share the same 4×4 grid (with the same edge removed), and the 6-, 7-, and 8-machine layouts share the same 5×5 grid (with the same two edges removed). The five differ from one another in the number of machines and their *placement*, not in network geometry (this is how the original published data are organized). Therefore, when Section 6.6 analyzes this batch, **the number of independent samples along the network-geometry dimension is 2, not 5**, whereas along the machine-placement dimension it is 5; all claims below are stated according to this division.

This batch differs from the original data in three respects. **First, the per-edge travel times are not original data.** Lyu published per-edge durations only for a single example instance, and those values are not uniform; the per-edge durations of the Appendix A layouts were never published. We therefore fill them in with "all edges equally weighted," which is structurally consistent with the assumption of a "constant unit-step duration" in the van Os model and differs only in the scale of the time unit. Consequently, the results of this batch are not compared with the reference values of Lyu or van Os; the batch serves only as a set of topologies not designed by us. **Second, the load and unload stations are merged.** The original layouts have separate loading and unloading stations, whereas our setting has a single load/unload station; we therefore take the loading station at the vehicles' starting location as the load/unload station and treat the unloading station as an ordinary grid node. **Third, their job data are not used.** In the public instance families we have measured, the average transport time is only a few percent of the average processing time, so transport hardly causes any contention; if their job data were also adopted, corridor contention and all phenomena that depend on it would become unobservable. Hence only the network is externalized. Another family of layouts shipped with the same toolkit (labeled Liu 2023 in its directory) is not included in this batch: these layouts allow diagonal moves, whereas our corridors are 4-connected; supporting diagonal moves would require modifying the lower-level router, not just the instances.

Disturbances. Unless otherwise noted, $t_{\text{now}} = 0.35\, C_{\max}(S^0)$. E1–E3 and E5 impose a single-window blockage on a busy corridor; E6 adds three further disturbance types at the same t_{now}; E4 blocks the corridors in the LU cut (Section 6.6); the disturbance-magnitude sweep sorts future operations by their earliest future reservation and takes the reservation windows of the first fraction φ as micro-blockages (without merging them).

Baseline methods. R1: scoping on the affected set of the operation-precedence graph + level-1 rerouting, with $\theta = 2$ throughout (this value affects only the size of $|U_{R1}|$ under Class A disturbances; by Proposition 1, any finite θ yields an empty set under Class B). Its scoping rule is taken from the operation-level local rescheduling approach described in Section 2.1 and is reimplemented by us. The correspondence is as follows. The scoping rule "take the directly disturbed operations as the affected set, then expand forward along the within-job operation precedence" corresponds to AOR [1, 16]. The two further components, *truncating the expansion at θ hops* and *mapping the operation set to the corridor reservations to be released*, are specified by us—the original setting has no reservation table, so this mapping has no counterpart there. The rerouting and replay components share the same implementation as R2, so that the two differ only in the scope object. R1 is therefore a baseline that transplants this scoping rule into our reservation setting; its results reflect how the rule performs in this setting, not how the original method performs in its own setting. R2/STRC: scoping on the reservation impact set + level-1 rerouting (scope escalation after failure is enabled in the magnitude sweep and disabled in E3/E5). R0+: global rescheduling that searches again within a time limit, starting from the original schedule—a genetic algorithm with population size 40, the chromosome of the original schedule seeded into the initial population, and the same fixed-prefix decoding as RD (occupancies before t_{now} are not rewritable). *No generation limit or early-stopping threshold is set*; the only termination condition is the wall-clock budget, whose values are given in Section 6.7. RS: global right shift, which neither scopes on the impact set nor reroutes (Section 6.9). RD: re-decoding of the original chromosome, the cheapest global recomputation in our implementation (same section). RA: no restriction to the impact set but rerouting retained, that is, all unfinished occupancies after t_{now} are made rewritable; everything else is identical to R2, so that the role of the reservation impact set is tested in isolation (same section). These five baselines are organized along "whether the scope is delineated × the recomputation range": R1 and R2 differ only in the scope *object*; RA and R2 differ only in *whether* the scope is delineated (the rerouting procedure is identical); RS neither delineates the scope nor reroutes but pushes everything back; RD and R0+ recompute globally without scoping on the impact set and differ only in how many generations they search.

Budget convention. The methods are not compared at the same budget level. R1, R2, RS, RD, and RA are each a single deterministic repair that terminates when the run finishes; their runtime is determined by instance size and impact-set size rather than by a budget setting. Only R0+ terminates on a wall-clock budget. All results involving R0+ are therefore results under a given budget; the four-method comparison in Section 6.9 is the one made within the same budget level (each method runs once and terminates). Wall-clock times are reported per batch and are not compared across batches.

Switch for scope escalation after failure. Scope escalation after failure is disabled in E3. If it were enabled, R1 might restore feasibility after escalating to all unfinished occupancies, and the measurement would capture the fallback capability of the escalation rule rather than the difference between scope objects. Escalation is effective in itself (in the magnitude sweep it restores feasibility in all cells), but it is unrelated to the variable under test, so it is disabled in the boundary ablation. Its end point is exactly RA in Section 6.9, where it is measured cell by cell as an independent baseline, including its cost.

Division of labor between the two batches. The two batches are not two samples of the same body of evidence; they answer different questions. The main batch answers *whether the phenomena exist and whether the mechanism can be identified in isolation*: under Class B disturbances, scoping on the operation-precedence graph yields an empty scope while the original schedule is infeasible (Section 6.2); the reservation impact set contains that scope and the fields outside the set remain invariant (Section 6.3); and only the scope object is replaced under the same rerouting procedure (Section 6.4). All three require full control over the instance parameters, especially the transport-to-processing

time ratio that determines whether corridor contention arises, and the public instance families cannot provide such control (Section 7.1, Limitation 4). The second batch answers a different question: *whether these phenomena hold only on layouts we chose ourselves*. It is used mainly for reproduction (Section 6.12) and yields no new performance-comparison conclusions.

The two batches are analyzed jointly in only two places. The first is E4. The three same-size layouts of the main batch belong to one family with very small geometric differences, so two explanations—"the indicator is too coarse" and "the instances differ too little in geometry"—cannot be distinguished. Conversely, without a self-built baseline matched parameter by parameter, the external layouts alone could not tell whether the variation in their impact-set share is large enough. The second batch accordingly narrows the conclusion on structural predictability. The second is E5: the E5 protocol and parameters are identical in both batches, and pooling serves only to increase the number of cells without changing the direction of the conclusions. The conclusions of all other sections are drawn from the main batch alone; the second batch only strengthens their external validity.

Statistical convention. The paper contains two inferential tests, both in Section 6.9: the difference between R2 and RA in rewrite fraction (the primary test) and the difference between R2 and RD in makespan. Both use the two-sided Wilcoxon signed-rank test, paired on the (instance, random seed) key defined above. After Holm correction for these two tests, the significance conclusion of each is unchanged. All other cell-wise win/loss counts, means, and medians are *descriptive* statistics, and no significance claims are made for them. The effect size accompanying the primary test is reported as a Hodges–Lehmann estimate of 0.037, with a 95% bootstrap confidence interval of $[0.018,\ 0.074]$ (percentile method, 10^4 resamples, fixed random seed); the mean paired difference is 0.064, with interval $[0.039,\ 0.093]$. Dispersion is reported as the interquartile range: the rewrite fraction has an IQR of 0.073 for R2 and 0.062 for RA. Because the two methods tie exactly in 18 of the 50 cells, the lower end of the confidence interval for the *median* paired difference is 0; this shows that ties make up about one third of the win/tie/loss counts. The tables of all other batches give only means, medians, and counts, without intervals.

6.2 E1: the operation-precedence graph yields an empty scope

This section examines only what scope each of the two scoping rules yields under Class B disturbances; it does not compare repair quality. Table 4 and Figure 5 (left) summarize the extended results for Proposition 1 under the corridor-blockage protocol of Section 6.1: in 50/50 cells, $T_{\text{impact}} = 0$, the seed set and the reservation impact set are nonempty, and the original schedule is infeasible after the blockage.

Table 4 merges the three conditions into the column "All three hold" (cell-wise results are in the `n_T_impact` and `feasible_after_block` columns of `e1_miss.csv`). Here $T_{\text{impact}} = 0$ follows directly from the convention of Definition 2 for Class B; the substantive result is that the latter column is always false, that is, the original schedule *is* indeed infeasible. Together, the two constitute a counterexample.

The absolute value of $|\mathrm{Cl}|$ grows with the instance (16.9, 48.4, 82.8–88.3), a direct consequence of the larger total number of occupancies. By contrast, $\mathrm{Cl}/|R|$ lies within 0.56–0.59 on all five instances and *fails* to distinguish the three `S8x4x4` layouts. This agrees with the conclusion of Section 6.6: the experiments yield no structural indicator that reliably predicts the impact-set size. In terms of contributions, this section supports C1: under Class B disturbances, scoping on the operation-precedence graph yields an empty scope although the original schedule is already infeasible.

Table 4. E1: the operation-precedence graph yields an empty scope under corridor blockage (10 random seeds per instance, 50 pairs in total). $|R|$ is the number of occupancies in the whole table, and $\mathrm{Cl}/|R|$ is the share of the reservation impact set in the whole table. Means are taken over random seeds. Dispersion is reported as the interquartile range (IQR): the per-instance IQR of $\mathrm{Cl}/|R|$ lies between 0.024 and 0.049; over the 50 pooled pairs, the median is 0.571 and the range is 0.500–0.655. The between-instance variation of this share is thus of the same order as its between-seed variation, and the means should not be read in isolation. "All three hold" is the number of cells in which the three conditions of Proposition 1 hold simultaneously ($T_{\mathrm{impact}} = 0$, a nonempty seed set, and an infeasible original schedule after the blockage). The last three rows share the same size and flexibility and differ only in corridor topology.

Instance	Seeds	All three hold	Mean $\lvert R\rvert$	Mean $\lvert\mathrm{Src}\rvert$	Mean $\lvert\mathrm{Cl}\rvert$	$\mathrm{Cl}/\lvert R\rvert$
`example_3x3x2`	10	10/10	28.6	5.6	16.9	0.59
`congested_8x4x4`	10	10/10	85.0	15.0	48.4	0.57
`S8x4x4 (high)`	10	10/10	150.5	15.5	88.3	0.59
`S8x4x4 (funnel)`	10	10/10	149.1	15.5	85.4	0.57
`S8x4x4 (mid)`	10	10/10	147.6	15.0	82.8	0.56
Total	50	50/50	—	—	—	—

Table 5. E2: containment (10 random seeds per instance; the level-1 feasibility column uses no scope escalation). E2a is structural closure (leakage = 0); E2b is the invariance of the corridor, time interval, and vehicle of *every* occupancy outside the set.

Instance	E2a closed	Level-1 feasible	E2b outside set invariant per occupancy
`example_3x3x2`	10/10	10/10	10/10
`congested_8x4x4`	10/10	10/10	10/10
`S8x4x4 (high)`	10/10	10/10	10/10
`S8x4x4 (funnel)`	10/10	10/10	10/10
`S8x4x4 (mid)`	10/10	10/10	10/10
Total	50/50	50/50	50/50

6.3 E2: containment

Structural closure (E2a) holds in 50/50 cells (leakage = 0), consistent with Proposition 2. Level-1 repair (without scope escalation) is feasible in 50/50 cells. The corridor, time interval, and vehicle of every occupancy outside the set remain unchanged (E2b, the observable form of Proposition 3), which likewise holds in 50/50 cells (Table 5).

E2a and E2b mean different things. E2a is a property *of the graph*, guaranteed by Proposition 2, and serves to check that the implementation agrees with the definition. E2b is a property *after repair*; it tests whether the four premises of Proposition 3 hold under our protocol. Premise (ii) requires that every occupancy outside the set be pre-committed successfully; if an occupancy outside the set overlaps a no-entry window, the pre-commit fails, which indicates that the seed set is incomplete (Remark 3). In terms of contributions, E1, E2, and the boundary ablation in Section 6.4 support the *object level* of C1 and C2: the reservation impact set is uniquely determined by the edge set, is closed under outgoing edges, and restores feasibility in every cell under the same rerouting procedure. The net benefit of scoping is given by the RA baseline in Section 6.9.

Remark 4 (An implementation defect misread as an edge-set defect). *If rerouting is decided per* operation *rather than per* occupancy, *E2b passes in only* 24/50 *cells, which is easily misread as "the dependency edge set fails to cover some weak couplings." The actual cause is a granularity mismatch: the rewritable set consists of* occupancies, *whereas*

under per-operation decisions the whole transport is replanned as soon as either its empty leg or its loaded leg falls in the impact set. When only the loaded leg enters the impact set, the empty leg of the same operation is also rewritten, even though its task label is not in the rewritable set; it is therefore recorded as drift outside the set. All 84 *drifting occupancies in the* 26 *failing cells are such same-operation empty legs, and* 71 *of them (*85%*) had already been executed before* t_{now}*, so rewriting them also violates Assumption A2. After switching to per-*leg *decisions (empty legs that have been executed or are not made rewritable keep their original plan, and only legs in the rewritable set are rerouted), the drift drops to zero. Hence what was measured was not how well the edge set covers the physical couplings, but the fact that premise (iii) of Proposition 3, "the router is invoked only for transports in* U*," was not satisfied by the implementation. The general form of this misjudgment is to misread a granularity mismatch between implementation and definition as a lack of expressiveness in the model.*

A mismatch of the same kind that E2b cannot detect. *Under per-leg decisions, a leg that* spans t_{now} *is still replanned as a whole: as soon as one of its segments ends later than* t_{now}*, the task label of the entire leg enters the rewritable set, so the segments it had* already traversed *before* t_{now} *are rescheduled as well. This amounts to sending the vehicle back to the start of the leg to drive it again, which violates Assumption A2. E2b cannot detect this case, because the task labels of these segments lie* inside *the impact set and are thus outside the scope of the comparison of fields outside the set. The case is detected by a separate audit of finished occupancies (comparing, one by one, the occupancies with* $t^e \le t_{\text{now}}$*): 36 of the* 50 *cells contain such rewrites,* 86 *in total. After refining the granularity to* segments *(segments already traversed are kept as they are, and rerouting starts from the node where the vehicle actually is at* t_{now}*), the audit drops to zero in all* 50 *cells, all pass rates are unchanged, and* $C_{\max}$ *improves in* 6 *cells and worsens in none. All tables in this paper report results of the segment-granularity implementation; the numbers in this remark (*24/50*;* 84 *occupancies in* 26 *cells;* 86 *occupancies in* 36 *cells) all come from the coarser-granularity implementations.*

6.4 E3: boundary ablation

Table 6 and Figure 5 (right) fix level-1 rerouting and *disable* scope escalation after failure. Under Class B disturbances, R1 has an empty rewritable set and is infeasible in 50/50 cells, whereas R2 is feasible in all cells.

The two arms use the same rerouting procedure, the same initial solution, the same t_{now}, the same random seeds, and the same blockage window; **the only difference is which occupancies are rewritable**. R1's rewritable set is empty, and level-1 rerouting is the identity operation on an empty set, so the original schedule is left unchanged and still occupies the blocked corridor. R1 therefore fails because it decides that no rescheduling is needed, not because of repair quality. The cell-wise runtime of level-1 rerouting has a median of 3.2 ms and a maximum of 12.8 ms; the last column of Table 6 is the mean over the 10 random seeds within each instance. The two statistics are defined differently and are not directly comparable. In terms of contributions, this section attributes the gap to the scope object (C2, C3): with the same rerouting procedure and only the scope object replaced, the feasibility rate changes from 0/50 to 50/50.

We separately test whether this contrast depends on the calibration of edge weights. Our corridor edges are equally weighted, so we rerun E1 and E3 on the same 50 pairs under three non-uniform edge-weight settings (`lognormal`, `lu_far`, and `split`; data in `weight_sweep.csv`). The results for "all three hold" and for the empty operation-precedence-graph scope are unchanged at every setting; R1 remains infeasible in all cells, and the numbers of level-1-feasible cells for R2 are 50/50, 50/50, and 49/50, respectively. The only infeasible cell occurs under the `split` setting (`S8x4x4_high`, random seed 1). The contrast therefore does not depend on the equal-weight calibration.

Table 6. E3: boundary ablation (no scope escalation; 5 instances × 10 random seeds). miss_B means that in the cell, R1's rewritable set is empty whereas R2's is nonempty. The last column is the mean over the 10 random seeds within each instance, which differs from the cell-wise median reported in the text.

Instance	miss_B	R1 feasible	R2 feasible	Mean $\lvert U_{R2}\rvert$	R2 runtime /ms
`example_3x3x2`	10/10	0/10	10/10	16.9	0.66
`congested_8x4x4`	10/10	0/10	10/10	48.4	2.84
S8x4x4 (high)	10/10	0/10	10/10	88.3	3.40
S8x4x4 (funnel)	10/10	0/10	10/10	85.4	5.09
S8x4x4 (mid)	10/10	0/10	10/10	82.8	3.96
Total	50/50	0/50	50/50	—	—

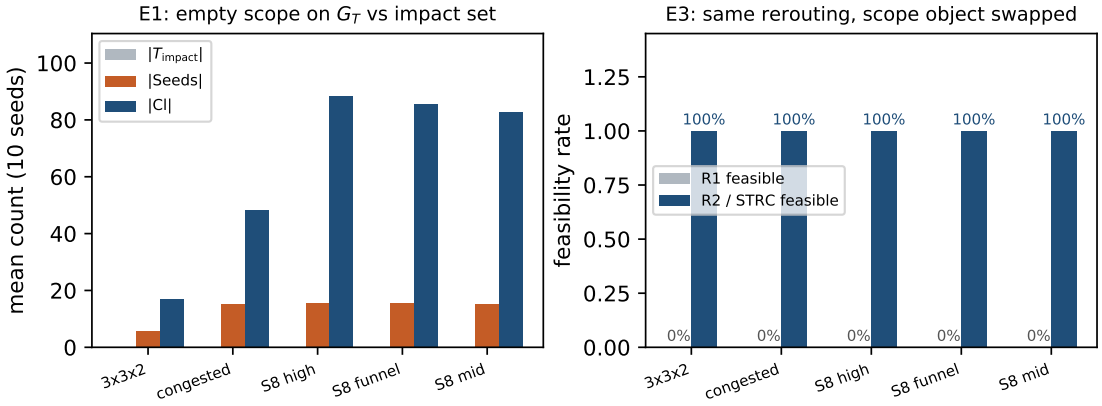


Fig. 5. Visualization of the extended E1/E3 results (means over 5 instances × 10 random seeds). Left: the affected set on the operation-precedence graph is empty, whereas the seed set and the reservation impact set are nonempty. Bar heights are means over the 10 cells of each instance; dispersion as IQR is given in Table 4 (the per-instance IQR of |Cl| lies between 2 and 8; the affected-set group is always zero and thus has no dispersion). Right: with the same rerouting and only the scope object replaced, the feasibility rate is 0 for R1 and 100% for R2; this panel shows cell-wise counts (50 cells each), so no dispersion is given.

6.5 Case Gantt chart

E1–E3 report counts and feasibility rates across random seeds. To show on the operation timeline a case that "the operation-precedence graph cannot capture but scoping on the reservation impact set can repair," Figure 6 takes one corridor blockage on `example_3x3x2` with random seed 42. The protocol is the same as in E2/E3: the decision time is $0.35\,C_{\max}$, a busy corridor is blocked in a single window, and level-1 rerouting is used without scope escalation.

In this case $T_{\text{impact}} = \varnothing$, whereas $|\text{Src}| = 6$ and $|\text{Cl}| = 18$. Figure 6(a) is the machine Gantt chart of the original schedule before the disturbance: blue operations have transport occupancies in the impact set (to be rerouted), and gray operations lie outside the impact set (pre-committed and kept as planned); the red dashed line marks t_{now}, and the light red band marks the blockage window. Figure 6(b) shows the schedule after repair: the operations in the impact set are rearranged on the timeline, and the schedule regains feasibility under the blockage, with $C_{\max} : 57 \longrightarrow 86$ and a sub-millisecond runtime. This figure only illustrates what kind of local change the statistical conclusions correspond to;

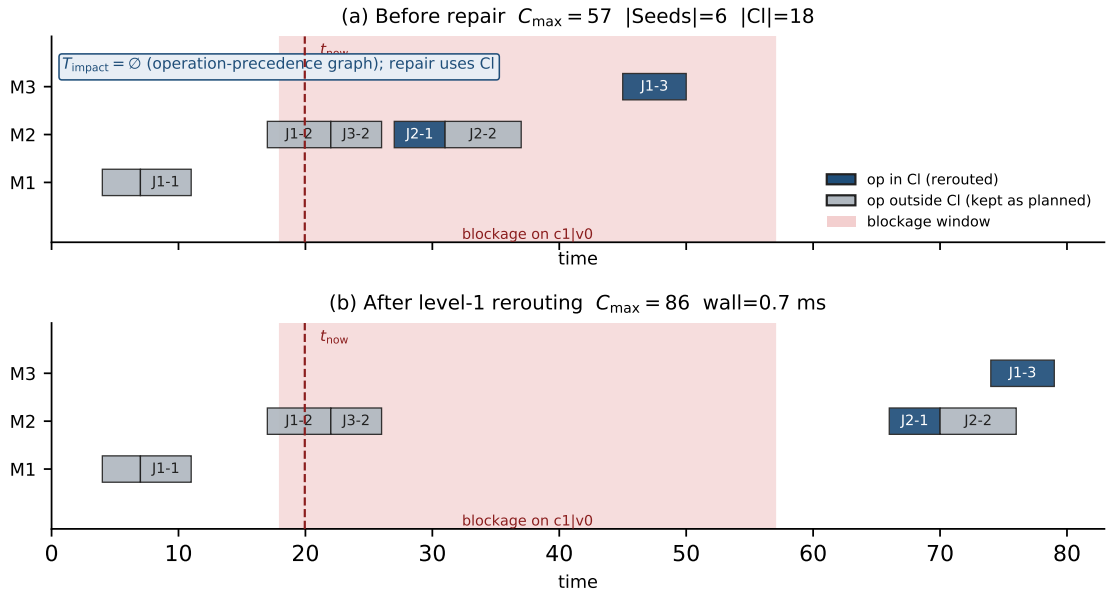


Fig. 6. Case Gantt chart (`example_3x3x2`, random seed 42). (a) Original schedule before the blockage, colored by membership in Cl; (b) after level-1 repair. The affected set on the operation-precedence graph is empty; blue operations are those whose transports in the impact set are rerouted. The figure only illustrates the form of the local change.

it is not used to compare makespans. The trade-off in makespan against global rescheduling is examined in E5 and in the disturbance-magnitude sweep.

6.6 E4: structure (exploratory)

When a corridor in the LU cut is blocked, the reservation-impact-set share of funnel and high fluctuates across random seeds (Table 7). In direction, funnel is higher (median 0.254 vs. 0.204; mean 0.244 vs. 0.229), but *the two groups overlap heavily*: funnel lies in $[0.180, 0.309]$ and high in $[0.156, 0.372]$, so the latter interval fully contains the former, and a single random seed suffices to reverse the direction (on random seed 2024, high reaches 0.372, above funnel's 0.296). With five random seeds and two layouts, this gap supports no conclusion. Under the busy-corridor blockage protocol of Section 6.2, the three equal-size layouts also yield $\mathrm{Cl}/|R|$ within the narrow band 0.56–0.59, and again fail to separate the layouts.

External layouts are used to separate the causes of the negative result. The three equal-size layouts all belong to the dumbbell family and differ little in geometry (Table 8: 9–11 nodes, 8–12 corridors, and an LU cut of only 1 or 2). On these layouts, two explanations cannot be told apart: "the metric is too coarse" and "the instances differ too little in geometry". The second batch of instances in Section 6.1 serves to separate the two (Table 8).

The results on the external layouts run opposite to the expected direction. This batch contains only 2 independent samples of network geometry (Section 6.1), so we use it only to examine the machine-configuration dimension. On *the same* 4×4 network, changing only the number and placement of machines from 4 to 5 drops the median impact-set share from 0.512 to 0.305, a range of 0.207. On the same 5×5 network, three machine configurations

Table 7. E4 reservation-impact-set share Cl/|R| (blocking a corridor in the LU cut; random seeds 42, 7, 2024, 99, 123). The instances in this table have 12 jobs / 8 machines / 12 vehicles, with the transport-to-processing time ratio calibrated to 4.0. **This setting differs from that of Table 8** (8 jobs / 4 machines / 4 vehicles, ratio 1.0, ten random seeds). Although both tables use the layout names `funnel/high`, *values are not comparable across the two tables*; only the rows within Table 8 form a like-for-like comparison.

Layout	42	7	2024	99	123	Median
funnel	0.254	0.181	0.296	0.309	0.180	0.254
high	0.156	0.204	0.372	0.229	0.184	0.204

(6, 7, and 8 machines) give a range of 0.075. For comparison, the three in-house layouts, which are *matched parameter by parameter* but use three *different* networks, give shares only within 0.446–0.497, a range of 0.051. Thus, **with the network fixed and the machine configuration varied, the impact-set share can vary more than it does across three different networks**: about four times as much on the 4×4 network and about one and a half times as much on the 5×5 network. The comparison is not symmetric. The external side changes both the number and the placement of machines, whereas the in-house side changes only the network; moreover, the three in-house layouts differ little in geometry, so 0.051 is only a *lower bound* on the effect of "changing the network". The comparison therefore shows that the machine-configuration dimension cannot be ignored, but not that its effect exceeds that of network geometry.

A minimum cut determined by the network alone cannot explain the size of the impact set. On these 5 external instances, **the LU cut is always** 2 **and the funnel share is always** 0. Both networks place the load/unload stations at grid corners, and a corner of a four-neighbor grid has exactly two outgoing edges, so the cut value is fixed at 2. The other layout family shipped with the same toolkit (excluded from this batch because it allows diagonal moves) places the load/unload stations at edge midpoints of the grid, where the out-degree is 3. **On real layouts, this metric therefore takes only a few discrete values determined by the grid positions of the load/unload stations**, rather than varying continuously with geometry. More importantly, **both metrics stay constant across all the above variations**, while the quantity to be explained varies by 0.207; a metric with a constant value cannot explain a varying quantity. The third metric (corridors per node) takes only two values over the whole batch (23/16 and 38/25), which is likewise insufficient to explain the within-group variation.

Hence, **on these 5 external instances and the three in-house layouts, static cuts do not explain the size of the impact set**. This family of metrics was designed for the tunable exit bottleneck at the load/unload stations of dumbbell layouts; on real layouts it degenerates into constants and has no explanatory power. The alternative direction thus points to the demand side: the size of the impact set may be determined not by how many paths the network *can* provide, but by *the spatial distribution of demand*; concrete candidate quantities are given in Section 7.2. This subsection reports an exploratory negative result and corresponds to none of the contributions.

6.7 E5: trade-off against global rescheduling

This subsection tests whether a *crossover* exists, that is, whether bounded repair dominates under tight budgets while warm-started global rescheduling overtakes it in $C_{\max}$ once the budget is relaxed. **No such crossover exists within the tested range of** 0.2–2 **s, and the conclusion extrapolates to larger budgets**. This follows from the shapes of the two curves. R2 is a single deterministic computation: the three budget levels of a cell share the same computation, so $C_{\max}$ does not vary with the budget. R0+ reports the best-so-far value per generation; with the random seed fixed, a larger budget only lets the same search trajectory run for more generations, so $C_{\max}$ is monotonically non-increasing. Hence, once the order of the two is settled at the *smallest* budget level, increasing the budget cannot reverse it, and only

Table 8. E4 comparison of in-house and external layouts (blocking a corridor in the LU cut, ten random seeds; median Cl/|R|). All rows are matched parameter by parameter and differ only in network and machine configuration. **The external instances lie on only 2 networks** (N_1: 4×4 with one edge removed; N_2: 5×5 with two edges removed). Rows within a group *share the same network* and differ only in the number and placement of machines. The "range" column is given *within each group*; the N_1 group has only 2 rows and the in-house group 3 rows, so ratios of ranges across groups indicate only the order of magnitude. The LU cut and the funnel share are *constant* over the entire external batch.

Source/ network	Instance	Machines	Nodes/ corridors	LU cut	Funnel share	Cl/|R|	Within-group range
External N_1	`LyuL2`	4	16/23	2	0	0.512	
External N_1	`LyuL3`	5	16/23	2	0	0.305	0.207
External N_2	`LyuL4`	6	25/38	2	0	0.385	
External N_2	`LyuL5`	7	25/38	2	0	0.422	
External N_2	`LyuL6`	8	25/38	2	0	0.460	0.075
In-house	`high`	4	10/10	2	0.286	0.497	
In-house	`funnel`	4	9/8	1	0.286	0.491	
In-house	`mid`	4	11/12	2	0.286	0.446	0.051

Table 9. E5 comparison with warm-started global rescheduling that does not rewrite finished occupancies (mean over 3 random seeds). "Rewrite fraction" is the share of occupancies in the whole table that are rewritten. This table covers only the in-house layouts: 2 instances × 3 random seeds × 3 budget points, i.e., 6 independent cells. Comparing the two $C_{\max}$ columns, R0+ is better at every budget point. Together with the external-layout batch (Section 6.12), there are 12 cells and 36 points, with a pointwise win/tie/loss of 35/1/0. R2 results do not vary with the budget, so the three rows of each instance share the same R2 result.

Instance	Budget/s	R0+ $C_{\max}$	R2 $C_{\max}$	Rewrite fraction R0+ / R2
`congested_8x4x4`	0.2	279.3	299.0	0.61 / 0.57
	1.0	276.0	299.0	0.61 / 0.57
	2.0	275.7	299.0	0.61 / 0.57
`example_3x3x2`	0.2	78.3	88.7	0.60 / 0.60
	1.0	78.3	88.7	0.60 / 0.60
	2.0	78.3	88.7	0.60 / 0.60

the smallest level needs to be examined. Among the 12 instance–seed cells, R0+ is strictly better in 11 cells, ties in 1, and is worse in none.

This monotonicity extrapolates only to *larger* budgets. At lower budgets, the order may reverse. R0+ needs at least one decoding per candidate evaluated (4.16 ms on the same batch, Section 6.9); given its population size, the wall-clock time needed just to evaluate the initial population is already of the same order as the smallest budget level, whereas R2 needs only 2.74 ms on the same batch and is feasible in all cells. With a still tighter budget, R0+ could not complete the evaluation of a single generation. The smallest tested level lies close to this boundary; smaller budgets were not swept.

Table 9 and Figure 7 present the in-house half of the data: two instances × random seeds $\{42, 7, 2024\}$ × budgets $\{0.2, 1, 2\}$ s. Section 6.12 runs the same protocol with the same code and the same parameters on two *external* layouts. The two batches together comprise 4 instances, 12 instance–seed cells, and 36 budget points; the pointwise R0+ win/tie/loss is 35/1/0. Both the monotonicity above and the budget invariance of R2 were checked cell by cell on all 12 cells. The 36 points are not independent samples (there are only 4 instances), so this subsection performs no significance test. The absence of a crossover follows from monotonicity; along the instance dimension, it is verified only on these 12 cells.

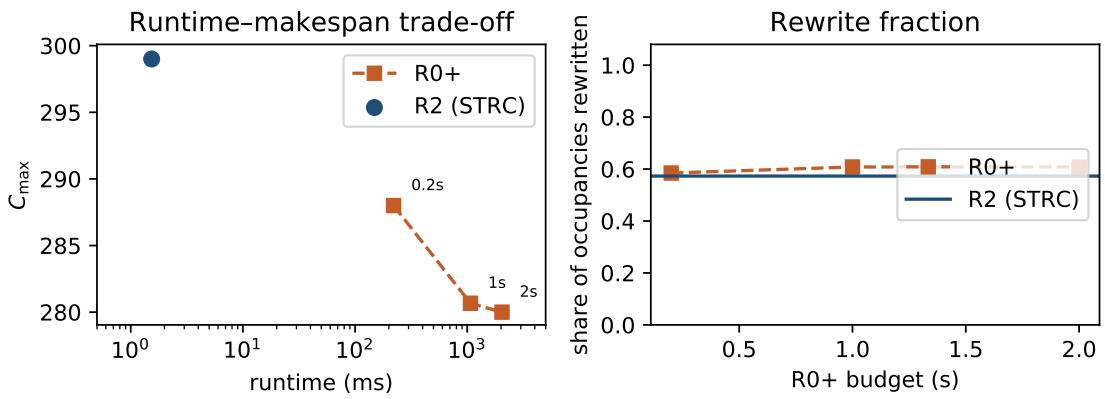


Fig. 7. E5: budget sweep of R0+ (which does not rewrite finished occupancies) versus bounded repair (congested_8x4x4, mean over 3 random seeds). Left: runtime versus C_{max}; the two curves do not intersect. Right: occupancy rewrite fraction, about 0.6 for both methods.

Bounded repair yields a worse makespan over the entire budget range, but its runtime is two to three orders of magnitude lower (3.2 ms for level-1 rerouting versus budgets of 0.2–2 s; see Figure 7, left). This runtime gap comes from a single rerouting pass that does not restart the search; it does *not* come from scoping on the impact set, as Section 6.9 confirms with the RA baseline of equal runtime. The rewrite fractions of the two methods are of the same order, 50%–66%; for a comparison of rewrite volume, see the RA comparison in Section 6.9. When a millisecond-scale response is required, bounded repair restores feasibility; when a response time of seconds is acceptable, warm-started global rescheduling that does not rewrite finished occupancies achieves a better makespan.

The R0+ in this table uses the fixed-prefix decoding of Section 5.3 and does not rewrite finished occupancies. A check for rewrites of finished occupancies found 0 violations at each of the 9 budget points of both the small and the congested instance. When a later operation of a job has already been carried away by a vehicle while an earlier operation still needs retiming, R0+ keeps the original vehicle and replans only the tail after t_{now}, thereby preserving the already executed empty-leg prefix. Bounded repair likewise has zero violations in the same cells. The makespan gap between the two therefore contains no contribution from "rewriting finished occupancies"; the remaining gap comes from the advantage of global reassignment and resequencing over a single rerouting pass. Consequently, the value of the reservation impact set does not lie in the makespan; its intended use is recovery that must respond within milliseconds.

6.8 E6: crossing disturbance types with scope objects

E1–E3 consider only one disturbance, corridor blockage. This subsection expands the disturbance-type dimension: on the same 50 pairs we add three further types, namely corridor slowdown, vehicle failure, and robotic-arm failure. The results are given in Table 10.

What is measured. For scoping, we report for all four disturbance types the size of the affected set on the operation-precedence graph, the size of the reservation impact set, and whether the latter contains the former. For repair, we run the repair for all four types. Class B still uses rerouting only: a blockage writes the no-entry window into the reservation

Table 10. E6 disturbance type × scope object (50 pairs per type, 200 measurements in total; the two Class B disturbances produce the same seed set by construction, so there are 150 *independent* scoping measurements; see the third point in the text). $|R^{\circ}|$ is the number of occupancies unfinished after t_{now}. "R1 empty" is the number of cells in which scoping on the operation-precedence graph deems the disturbance to require no rescheduling. The last column uses no scope escalation: level-1 rerouting for Class B, and vehicle swapping or machine reassignment for Class A.

Disturbance	Class	R1 empty	Mean $\lvert U_{R1}\rvert$	Mean $\lvert\text{Cl}\rvert$	$\text{Cl}/\lvert R^{\circ}\rvert$	R2 feasible
Corridor blockage	B	50/50	0.0	64.4	0.92	50/50
Corridor slowdown	B	50/50	0.0	64.4	0.92	50/50
Vehicle failure	A	0/50	31.1	60.6	0.85	50/50
Robotic-arm failure	A	0/50	42.8	65.9	0.94	50/50

table, while a slowdown reduces traversal progress by a factor of $\tau_{\text{mult}} = 2$ within the window, traversal outside the window follows the original τ, and the corridor remains passable. Class A additionally allows vehicle swapping or machine reassignment, without changing the processing order; whether the rewritable scope is escalated does not affect its action set.

The results comprise four points. First, **the A/B dichotomy holds without exception over the** 150 ***independent* scoping measurements** (the two Class B disturbances share the same seed set; see the third point): scoping on the operation-precedence graph is empty for both Class B disturbances in every case and nonempty for both Class A disturbances in every case. Second, **over all** 100 **Class A measurements,** $U_{R1} \subseteq \text{Cl}$; that is, on Class A, switching to scoping on the reservation impact set never *misses* an occupancy already included by the operation-precedence graph. The containment is strict in 87/100 cells, and the two sets are equal in the remaining 13 cells. For the robotic-arm failure row, containment holds *by construction*: the seed rule of Definition 3(c) takes the entire suffix along the job chain, whereas U_{R1} is truncated at $\theta = 2$ (Section 6.1); this row thus only verifies that the implementation agrees with the definition. For the vehicle failure row, containment is an empirical result. The price is a larger rewritable set on average: 31.1 $\longrightarrow$ 60.6 for vehicle failure and 42.8 $\longrightarrow$ 65.9 for robotic-arm failure. Third, **slowdown and blockage produce identical scoping results cell by cell**, with the same impact-set size on all 50 pairs. The reason is that the two share the same seed rule (unfinished occupancies overlapping the failure window), which by construction yields the same reservation impact set; the row shows that scoping on the operation-precedence graph also yields an empty scope for slowdown. Fourth, **for repair, all four disturbance types are feasible in every case**; per-type results are in the last column of Table 10, with slowdown measured on the same basis as blockage. A slowdown only scales the progress of the part overlapping the window, and the corridor remains passable, so it still differs physically from "full no-entry followed by a detour". Level-1 rerouting already achieves full feasibility, and scope escalation is never triggered. Vehicle failure is handled by vehicle swapping and robotic-arm failure by machine reassignment.

This table also provides an empirical measure of the impact-set size (Remark 2). Under corridor blockage, the reservation impact set covers about 92% of the occupancies unfinished after t_{now}; compared with the trivial upper bound of "including all unfinished occupancies in the rewrite", it is smaller by less than one tenth. This does not contradict the range 0.56–0.59 of $\text{Cl}/|R|$ computed over the whole table in Section 6.2: the whole table also contains the roughly half of the occupancies that were completed before t_{now} and cannot be rewritten. In terms of contributions, this subsection extends the conclusions of C1 and C2 from one disturbance type to four.

Table 11. E7 comparisons within the millisecond range (50 pairs; same protocol as E3: single-window blockage of a busy corridor, $t_{\text{now}} = 0.35\, C^0_{\max}$, R2 without scope escalation). All four methods are feasible in 50/50 cases. "Finished-occupancy violations" is the number of cells in which finished occupancies were rewritten; "Rewritable" is the share of occupancies unfinished after t_{now} that the method includes in the rewrite; the last column gives the method's $C_{\max}$ win/tie/loss against R2. RA and R2 differ only in which occupancies are rewritable, so the entire difference between these two rows is attributable to the reservation impact set. This table is *not* directly comparable with Table 9, which contains only two instances.

Method	Finished-occupancy violations	ms (median)	Mean degradation	Rewrite fraction	Rewritable	W/T/L vs. R2
RS global right shift	0/50	1.42	58.8%	0.627	—	7/16/27
RD re-decoding	0/50	4.16	49.0%	0.618	—	22/14/14
RA no impact-set scoping	0/50	2.76	49.9%	0.604	1.00	0/46/4
R2 bounded repair	0/50	2.74	49.8%	0.540	0.92	—

6.9 E7: comparisons within the millisecond range

The baseline in E5 is a warm-started population-based search with a budget of 0.2–2 s. Using it as the only baseline leaves a question open: *how much of the "two to three orders of magnitude lower" result is due to the reservation impact set, and how much merely to the longer runtime of the baseline?* This subsection adds baselines whose runtime lies between those of bounded repair and warm-started population-based search, together with a baseline that changes only the rewritable scope.

- **RS (global right shift).** Without changing routes, assignments, or processing orders, RS postpones all unfinished occupancies as a whole by a delay Δ, where Δ is just large enough to move every movable occupancy on the blocked corridor past the blockage window. Like R2, it does not rewrite finished occupancies (those with $t^e \le t_{\text{now}}$ are immovable). Because the shift is uniform, every time gap stays the same or grows, and all constraints are of the "$\ge$" type, so a feasible solution is obtained without rerouting. RS represents one extreme of "*not scoping on the impact set*" and performs no rerouting at all (Section 2.1).
- **RD (re-decoding of the original chromosome).** RD keeps the machine assignments and processing orders and runs the same fixed-prefix decoding as R0+ on the routing layer into which the blockage has been written. It is the *fastest global recomputation* in our implementation; because it does not change the chromosome, it is faster than R0+ but has less freedom to reassign and resequence.
- **RA (no scope restriction by the impact set, but with rerouting retained).** RA includes all occupancies *unfinished* after t_{now} in the rewrite and then runs a rerouting replay *identical* to that of R2. It represents the other extreme of "not scoping on the impact set": unlike RS, it retains rerouting, so its only difference from R2 is the set of rewritable occupancies. The two use the same rerouting procedure, neither rewrites finished occupancies, and both face the same blockage. RA is therefore the *only* baseline in this paper that isolates the effect of the reservation impact set; the difference between the two methods contains no contribution from the rerouting procedure, the protocol, or the implementation. Since RA's rewritable set is already maximal, it needs no scope escalation after failure.

E3 compares two *different* scope objects (the operation-precedence graph and the reservation impact set), whereas RA addresses the question *whether the rewritable scope needs to be restricted by the impact set at all.*

Table 11 yields four findings.

First, among the methods that return a feasible plan, the only one faster than bounded repair is the global right shift, but it is worse in both makespan and rewrite fraction. (R1, which scopes on the operation-precedence graph, is faster, but its feasibility rate is only 0/50, so it is not a candidate method.) RS needs only 1.42 ms, about half the time of R2. The price is a mean makespan degradation of 58.8% (versus 49.8% for R2), a cell-by-cell result of 27/16/7 (R2 win/tie/loss), and more rewritten occupancies (0.627 vs. 0.540), because the uniform shift changes the times of every unfinished occupancy. Making the right shift *selective*, that is, delaying only the vehicles actually affected and leaving the rest unchanged, would first require determining which occupancies are affected, which is precisely the scope this paper delineates. **The right shift needs no impact-set scoping precisely because it gives up selectivity.**

Second, even the fastest global recomputation is slower than bounded repair. The median runtime of RD is 4.16 ms, 1.5 times that of R2. This result does not depend on the search configuration: any decoding-based search needs at least one decoding per candidate evaluated. The "two to three orders of magnitude lower" result in E5 is therefore *not* an artifact of an overly slow baseline; with the baseline replaced by the fastest global recomputation in the same implementation, bounded repair is still faster.

Third, without rewriting finished occupancies, RD and R2 are indistinguishable in makespan on this sample, but RD rewrites more. Over the whole table, R2 versus RD is 14/14/22 (win/tie/loss), with mean degradations of 49.8% and 49.0%. The difference is not significant: the mean paired difference is +0.006 with a bootstrap 95% interval of $[-0.007,\ 0.019]$, which contains 0; the two-sided Wilcoxon signed-rank test gives $p = 0.12$ (36 cells with nonzero differences); and the mean $C_{\max}$ ratio is 0.997 with interval $[0.988,\ 1.007]$, which likewise contains 1. No makespan difference between the two is thus observed. RD has 0/50 finished-occupancy violations: when a later operation has already been carried away by a vehicle, RD keeps the original vehicle and replans only the tail after the decision time, thereby preserving the already executed empty-leg prefix. RD's slight edge in the cell-by-cell count therefore stems from the advantage of global rerouting under the same chromosome over a single rerouting pass, not from rewriting finished occupancies. RD's rewrite fraction is 0.618, higher than R2's 0.540; that is, the method that scopes on the impact set rewrites less.

Fourth, the reduction in rewrite volume can be established only by comparing R2 with RA, and it manifests only as stability. RA and R2 differ only in the set of rewritable occupancies, so this pair directly gives the net benefit of scoping on the impact set. On average, the reservation impact set includes only $1 - 0.92$ fewer unfinished occupancies than the trivial upper bound (0.80–1.00 per cell; in the 6 cells with value 1.00, the impact set *equals* the set of all unfinished occupancies, and the two methods give identical results). This difference of less than one tenth lowers the rewrite fraction from 0.604 to 0.540: a mean reduction of 0.064, a median reduction of 0.029, a cell-by-cell result of 31/18/1 (R2 more stable/tie/less stable), and a two-sided Wilcoxon signed-rank $p = 8.8 \times 10^{-7}$.

Excluding the 6 identical cells above leaves 44 cells, with a cell-by-cell result of 31/12/1 and a mean reduction of 0.073 (the Wilcoxon test has 32 cells with nonzero differences under both counts); the reduction reported over 50 cells is thus conservative. The direction of the reduction is highly consistent, but its *magnitude is small*; in the single counterexample cell, R2 rewrote 0.007 more. Meanwhile, the two methods have nearly identical runtimes (2.74 vs. 2.76 ms) and nearly identical makespans (mean ratio 0.999; cell-by-cell R2 win/tie/loss of 4/46/0, never worse, with 0.970 in the best cell).

These three results must be read together. **Equal runtime shows that the millisecond-scale response comes from a single rerouting pass that does not restart the search, not from the reservation impact set (results in Table 11; single-pass rerouting itself is an existing technique, see Table 2). Equal makespan shows that the impact set is not an optimization device. A significantly lower rewrite fraction shows that scoping delivers exactly the property guaranteed by Proposition 3: occupancies outside the set keep their corridors, intervals,**

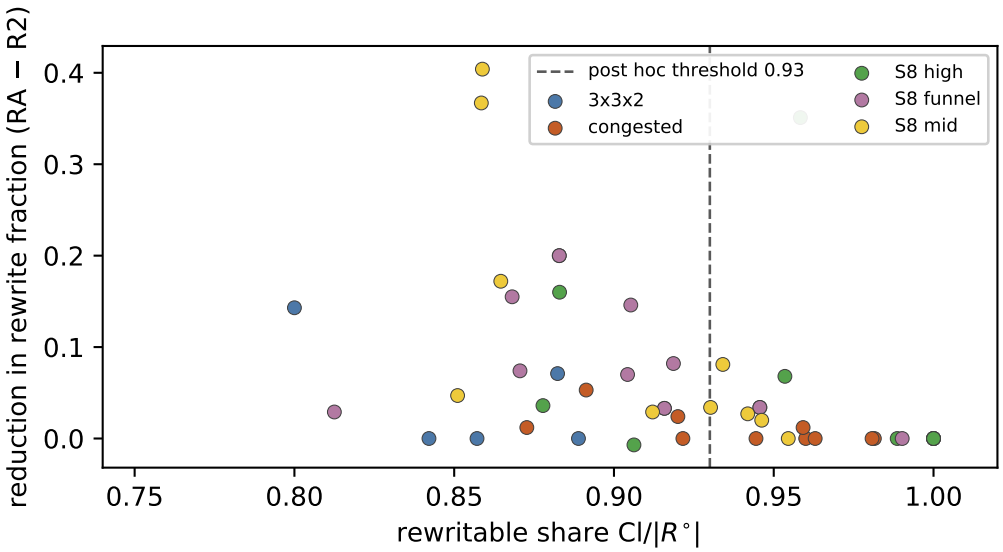


Fig. 8. E7 post hoc rule: rewritable share versus reduction in rewrite fraction (50 pairs, R2 vs. RA). The dashed line marks the in-sample threshold 0.93; positive values on the vertical axis mean a lower rewrite fraction with impact-set scoping.

and vehicles unchanged. The role of the impact set is to *mark which occupancies need not be rewritten*. The more occupancies the impact set excludes, the larger the reduction; however, the excluded occupancies themselves account for only part of the gap 0.604 − 0.540; the rest comes from changes in the rerouting outcome once the rewritable set shrinks. On the instance where the impact set nearly equals the set of all unfinished occupancies (`congested_8x4x4`, rewritable share 0.94), the reduction is only 0.010; on the instance that excludes more occupancies (`S8x4x4_mid`, 0.91), the reduction reaches 0.118. Two instances can only indicate a direction; the per-cell distribution is given below.

We next condense this direction into a post hoc threshold rule. Figure 8 plots the 50 pairs with the rewritable share $\mathrm{Cl}/|R^{\circ}|$ on the horizontal axis and the reduction in rewrite fraction of R2 relative to RA on the vertical axis. On this sample we take $\mathrm{Cl}/|R^{\circ}| \geq 0.93$: the 23 cells above this line have a mean reduction of 0.029, and the 27 cells below it have a mean reduction of 0.094. All cells with a share above 0.97 show zero reduction, but for two different reasons. In the 6 cells whose share is exactly 1.00, the impact set equals the set of all unfinished occupancies, so the two methods are *identical*; in the remaining cells, the share is below 1.00 yet the reduction is already 0, which is an empirical result rather than an identity. The threshold is chosen on this sample to answer "when does scoping on the impact set still pay off"; it is not meant to be extrapolated, and it differs from a structural prediction of the impact-set size (Section 6.6).

Combining the results of the four methods: under the two constraints that "finished occupancies are not rewritten" and that "the response must be within milliseconds", **the net benefit of scoping on the impact set appears only in the rewrite volume. At equal speed and makespan, forgoing scoping rewrites about** 6 **percentage points more occupancies (a relative increase of about** 12%**)**, whereas scoping brings *no* advantage in speed or makespan. When the runtime constraint is relaxed, a shorter makespan still comes from warm-started search (Section 6.7), but its runtime then exceeds the millisecond range.

Table 12. Comparison across disturbance magnitudes (congested_8x4x4, averaged over 10 random seeds). Bounded repair includes scope escalation; R0+ budget 2 s. Dispersion is reported as the interquartile range: over the whole sweep, the STRC runtime ranges over 2.2–23.8 ms, and the per-level IQR of the R0+ $C_{\max}$ lies between 8 and 76. **At the two smallest levels, the within-level IQR of the "Time ratio" column exceeds the between-level variation of that column, so the column indicates only the order of magnitude, not a trend in φ.** The column gives the level mean of *per-cell* ratios (paired first, then aggregated); it therefore does not equal the ratio of the level means of the two adjacent columns. By the convexity of $1/x$, the two generally differ, and recomputation should use the per-cell data. The numerator is the wall-clock *budget* of R0+, not its measured runtime, so this is not a comparison at equal quality (Section 6.1).

φ	STRC feasible	STRC ms	R0+ $C_{\max}$	Time ratio
0.10	100%	9.1	103.5	406×
0.25	100%	11.1	107.0	325×
0.50	100%	8.6	111.8	333×
0.75	100%	8.4	125.8	254×
1.00	100%	10.7	148.1	203×

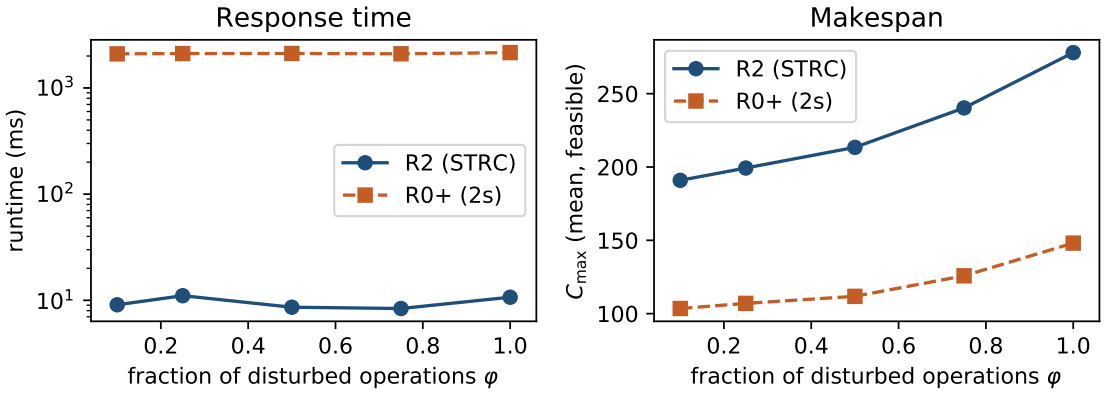


Fig. 9. Sweep over disturbance magnitude (congested_8x4x4, 10 random seeds): runtime (log scale) and makespan of bounded repair and R0+ as functions of φ. Curves show per-level means; dispersion is measured as in Table 12 (interquartile range). On the runtime side, the range over the whole sweep is 2.2–23.8 ms, always on the order of ten milliseconds; this is the only result on which the conclusion of this section rests.

6.10 Sweep over disturbance magnitude

Table 12 and Figure 9 report the sweep over φ (congested_8x4x4, 10 random seeds; R0+ budget 2 s). Per-level results are given in the table; R0+ still attains the better $C_{\max}$ throughout, consistent with Section 6.7. On the small instance example_3x3x2, every cell is likewise feasible (10 random seeds × five values of φ); bounded repair takes on the order of 1 ms, and the time ratio ranges from 87× to 3051× (per-cell extremes across both instances). The numerator of this ratio is the *budget* of R0+ and the denominator is the measured wall-clock time of bounded repair. Since R0+ attains the better $C_{\max}$, the ratio should not be read as a speedup at equal solution quality.

Bounded repair in this section has scope escalation after failure enabled, so its results are not directly comparable with those of E3 and E5. Even when the disturbance fraction reaches $\varphi = 1$ (all future operations disturbed), the runtime remains on the order of ten milliseconds and does not grow noticeably with the disturbance magnitude. This runtime is due to single-pass rerouting (Section 6.9).

Table 13. E8 sweep over instance size (7 levels × 10 random seeds). "Impact set" is the reservation impact set as a share of the *whole table*; the attainable upper bound of this share is $|R^\circ|/|R|$, whose mean in this batch is about 0.61. With the rewritable set as the denominator, the share is 0.78–0.91 (see the text). Runtimes are medians, and the "RD/R2" column is the *ratio of the two median columns*. When the text discusses monotonicity, it uses level means of per-cell paired ratios; the two measures rise and fall alike across levels but differ in value. The last column gives the win/tie/loss of R2 against RS on $C_{\max}$.

Size	$\lvert R\rvert$	Impact set	R2 ms	RD ms	RD/R2	R2 vs. RS
$8 \times 4 \times 4$	148	0.561	2.9	4.4	1.5×	7/1/2
$10 \times 5 \times 5$	172	0.485	3.2	6.1	1.9×	9/0/1
$12 \times 6 \times 6$	230	0.481	4.5	10.7	2.4×	7/1/2
$14 \times 7 \times 7$	281	0.492	6.1	14.5	2.4×	8/0/2
$16 \times 8 \times 8$	291	0.534	6.4	16.4	2.6×	10/0/0
$18 \times 9 \times 9$	356	0.535	8.2	26.4	3.2×	10/0/0
$20 \times 10 \times 10$	402	0.476	10.4	34.2	3.3×	10/0/0

6.11 E8: extrapolation to larger instances

The previous section swept the magnitude of the *disturbance*, with the instance fixed at 8 jobs and 4 machines. This section turns to the other dimension: the disturbance protocol is fixed and the instance itself is enlarged. The key question here is not runtime (runtime is expected not to grow noticeably) but **whether the impact-set share approaches 1 as the instance grows, which would make "bounded" meaningless in practice**.

The size sequence uses $2k$ jobs, k machines, and k vehicles for $k = 4, \ldots, 10$, that is, from $8 \times 4 \times 4$ to $20 \times 10 \times 10$: 7 levels × 10 random seeds = 70 pairs. The congestion level, heterogeneity, flexibility, transport-to-processing time ratio, and the load/unload-station minimum cut and remote cut are all calibrated to the same targets, so *only the size changes*. The three baseline methods are defined as in Section 6.9.

Per-level results are given in Table 13.

The impact-set share does not rise with instance size, but whether it "approaches 1" can only be answered with the appropriate denominator. Across the seven levels, $\mathrm{Cl}/|R|$ fluctuates between 0.48 and 0.56 without monotone trend (highest at the first level, lowest at the last, and back above 0.53 at the fifth and sixth). Because Cl contains only occupancies that are unfinished after t_{now}, and the mean of $|R^\circ|/|R|$ in this batch is only 0.61, the attainable upper bound of $\mathrm{Cl}/|R|$ is about 0.61, not 1. With the rewritable set as the denominator, $\mathrm{Cl}/|R^\circ|$ lies in 0.78–0.91 across the seven levels (per-cell maximum 1.00), the same order of magnitude as the 92% of the main batch in Sections 6.8 and 6.9. This section therefore shows that *the share does not rise with instance size*, but the impact set is not small relative to the rewritable set (Remark 2), and seven levels are insufficient to support an asymptotic conclusion.

Runtime grows faster than the total number of occupancies. As $|R|$ grows by a factor of 2.7, the median runtime of R2 grows from 2.9 ms to 10.4 ms, a factor of 3.6, and remains on the order of ten milliseconds; seven levels are insufficient to determine the growth order. Level-1 rerouting is feasible in 69/70 cells. The single failing cell ($10 \times 5 \times 5$, random seed 42) restored feasibility in round 3 of scope escalation after failure, with a total runtime of 12.0 ms and the same makespan as the level-1 attempt; hence the final number of feasible cells is 70/70.

The runtime gap between the two baseline methods widens with instance size. The runtime ratio of the fastest global recomputation to bounded repair rises from 1.5× to 3.3×; that is, the larger the instance, the higher the runtime cost of global recomputation. The per-level ratio is *not monotone* (the level mean of per-cell paired ratios dips from $k = 6 \to 7$), but the two ends differ by 1.8, far more than any within-level interquartile range (at most 0.74); the *overall rise* is therefore not noise.

Against the global right shift, which does not delineate the scope by the impact set, R2 has a $C_{\max}$ win/tie/loss of 10/0/0 at each of the last three levels, while the first four levels range between 7 and 9 wins with no monotone change; with only 10 cells per level, no trend can be established.

Scope of the evidence. This size sequence covers a single congestion level and a single instance seed (every level's instance is generated from random seed 42), and tests only the dimension of *size*; the sweeps of Sections 6.8 and 6.10 were not rerun on this batch. In terms of contributions, this section adds evidence along the size dimension to the boundedness claim of C2: the share did not rise across these seven levels.

6.12 Reproduction on the external-layout batch

For the second batch of Section 6.1, whose layouts are taken from an external source, the results of E1, E2, E3, and E5 on 50 instance–seed pairs point *in the same direction, item by item,* as those of the main batch: the operation-precedence graph yields an empty scope in 50/50, structural containment (i.e., zero leaks) holds in 50/50, and fields outside the set are unchanged entry by entry in 50/50. Under the boundary ablation, level-1 rerouting is feasible in 0/50 cells when scoping on the operation-precedence graph and in 50/50 when scoping on the reservation impact set. Under the E1 protocol, the per-layout medians of the impact-set share lie in 0.438–0.547; the corresponding main-batch result is the pooled median over 50 pairs, 0.571 (0.56–0.59 is the range of per-instance *means*, a different measure). The external batch is slightly lower overall, but both are about one half, and the shortfall is of the same order as the variation across instances (figure caption in Section 6.2). The trade-off against global rescheduling is also unchanged: at 17/18 budget points R0+ attains the better $C_{\max}$, at the remaining 1 point the two methods tie (`LyuL6_8m`, random seed 2024, budget 0.2 s), and at no point is R0+ worse. This batch uses the same E5 code and the same parameters as the main batch, so Section 6.7 analyzes the two batches pooled over 12 instance–seed cells; the tied cell also satisfies the monotonicity criterion of that section, and there is still no crossover.

This batch tests whether the above phenomena hold only on layouts designed by us. The results also hold on a batch of topologies not chosen by us, so the object level of C1 and C2 does not depend on our layout design. The only conclusion this batch changes is the one on structural predictability (Section 6.6).

6.13 Summary

We group the results of this section by the contribution level they support; the limitations and the levels they affect are collected in Section 7.1.

The object level (C1, C2: on which object the scope is delineated, and whether the resulting set is uniquely determined) is supported by four groups of results. Under corridor blockage, the scope on the operation-precedence graph is empty although the original schedule is indeed infeasible (E1, 50/50 over 50 pairs). The reservation impact set is closed under dependency edges, and fields outside the set are unchanged entry by entry (E2, 50/50, 50/50). With the same rerouting procedure and only the scope object changed, the feasibility rate goes from 0/50 to 50/50 (E3), so feasibility depends on the scope object rather than on the quality of the rerouting procedure. Across four disturbance types crossed with two scope objects, the A/B dichotomy holds without exception, and for Class A the reservation impact set always contains the scope obtained on the operation-precedence graph (E6). Propositions 1–3 give the corresponding structural statements, with Proposition 3 still serving only as a sufficient condition.

The net-benefit level (C3: what this way of delineating the scope gains) is supported by a single group of results. Switching to a baseline that does not restrict the scope to the impact set (RA, which uses the same rerouting as R2 and likewise does not rewrite finished occupancies) leaves runtime and $C_{\max}$ almost unchanged, yet raises the rewrite

fraction from 0.540 to 0.604 (per cell 31/18/1, $p = 8.8 \times 10^{-7}$; Section 6.9). Hence the millisecond-scale response is due to single-pass rerouting rather than to the impact set, and **the reduction in rewriting can be attributed only through this comparison**. Compared with warm-started global rescheduling, bounded repair is better on $C_{\max}$ in no cell, and relaxing the budget produces no crossover (E5); E7 rules out the explanation that the baseline methods simply ran too long.

The remaining results are qualifying results. The instance size has been extended to 20 jobs and 10 machines, and the impact-set share with the rewritable set as denominator did not rise with size (E8); structural predictability is a negative result (E4). The second batch of instances (50 pairs) only tests whether the above phenomena depend on layouts designed by us, and the results of E1, E2, E3, and E5 are reproduced item by item.

7 Conclusion

This paper makes three contributions. **First** (C1), it identifies a class of failures that the operation-precedence graph cannot capture: corridor blockage changes no operation assignment, so the scope delineated by affected operations is empty, although the original schedule is indeed infeasible (50/50 over 50 pairs, Section 6.2). **Second** (C2), it takes the reservation impact set obtained by STRCas the rescheduling scope: once the dependency relations are given, whether an occupancy belongs to the set is uniquely determined, and no dependency edge points from inside the set to outside it (50/50, 50/50, Section 6.3). **Third** (C3), it provides a local recovery matched to this scope and separates the sources of the individual results: with the same rerouting procedure and only the scope object changed, the feasibility rate goes from 0/50 to 50/50 (Section 6.4), so feasibility depends on the scope object rather than on the quality of the rerouting procedure.

Yielding relations, rerouting inside the set while keeping everything outside it as planned, and scope escalation after failure are all taken from the existing literature (Section 2.5). Our claim is as follows. By definition, the operation-precedence graph contains no yielding relations between corridor occupancies, and therefore cannot capture disturbances that invalidate only occupancies. Once the scope is delineated on the reservation dependency graph, the occupancies that must be rewritten are uniquely determined by the given precedence relations, not by a heuristic radius.

If the rewritable set is changed from the reservation impact set to "all occupancies unfinished after t_{now}" with all else unchanged, runtime and makespan remain almost the same, yet the rewrite fraction rises from 0.540 to 0.604 (Section 6.9). Hence the millisecond-scale response comes from single-pass rerouting without restarting the search; **the reduction in rewriting can be attributed only through this comparison, and it reflects the property guaranteed by Proposition 3: occupancies outside the set are kept as planned**. The size of this benefit depends on how much smaller the impact set is than this trivial upper bound, and the benefit vanishes as the two approach each other.

7.1 Limitations

The following four limitations are ordered by their impact on the preceding results; Section 7.2 gives a direction for addressing each of them.

(1) **The makespan is worse than that of warm-started global rescheduling that does not rewrite finished occupancies, and relaxing the budget produces no crossover.** Bounded repair trades makespan for runtime, and over the 12 instance–seed cells tested, this disadvantage does not reverse as the budget is relaxed (Section 6.7).

This limitation concerns the net benefit of the scoping, not the object-level conclusions; the comparison of rewriting volume is given in Section 6.9.

(2) **For the reservation impact set, the guarantee of Proposition 3 is only a sufficient condition; the dependency edge set has not been independently and exhaustively verified; the rewritable set is not tight; and its size cannot be predicted by static indicators that depend only on the network.** The impact set covers about 92% of the occupancies unfinished after t_{now} (Remark 2). The size of the net benefit depends on how much smaller the impact set is than this trivial upper bound, and the benefit vanishes as the two approach each other. The family of static cut indicators has been ruled out in Section 6.6, and the general relation between network geometry and impact-set size remains open. This limitation concerns the strength and tightness of the object-level guarantee; it does not affect the diagnosis that scoping on the operation-precedence graph yields an empty scope.

(3) **The implementation covers less than the problem scope described in the text: Class B disturbances are handled by rerouting only; Class A additionally allows vehicle swapping and reassignment but no change of processing order; and, by Assumption A3, only a single response is made.** Opening up the processing order degenerates into global rescheduling, at which point "bounded" loses its meaning; disturbance streams, prediction errors, and rolling responses are not addressed. The size dimension has been extended to 20 jobs and 10 machines without the impact-set share rising with size, but the number of levels swept is insufficient to support asymptotic claims. This limitation restricts extrapolation of the results to larger instances and to continual disturbances.

(4) **External reference points are lacking: this line of research has no public benchmark that can be substituted in, the provenance of the instances is externalized only at the topology level, and the results have not been reproduced on a second implementation.** The public instance families [3, 6] publish transport times as machine-to-machine travel-time matrices rather than as networks. We checked all 11 layouts of the two families one by one, and none of them can be reconstructed as an undirected corridor graph consistent with the published matrix; hence R, G_R, $\mathrm{Cl}(\cdot)$, and Class B disturbances cannot be defined on them (Section 6.1). The second batch of instances is built from the appendix layouts of Lyu et al. [10], so the layouts are no longer designed by us. However, the per-leg travel times and job data are still our own, so the batch cannot be used to reproduce the reference values of Lyu et al. or van Os [20], and its coverage is also smaller than that of the main batch. This limitation concerns external validity; the relative results among baselines within the same implementation are unaffected.

7.2 Future work

The following four directions correspond one to one to the four limitations of the previous section.

1. Narrowing the makespan gap (Limitation 1). In the main batch, prefix decoding does not rewrite finished occupancies (the number of violations over the 18 budget points of E5 is 0, and that of RD on E7 is 0/50), so Limitation 1 concerns only $C_{\max}$ itself. Narrowing the gap would require limited reassignment or resequencing without rewriting finished occupancies, but this would make the method degenerate toward global rescheduling.

2. Making the guarantee of the reservation impact set more complete and tighter, and characterizing its size (Limitation 2). *Completeness*: under the current yield edge set, the leakage found by the probe is 0. The next step is to probe with other disturbance forms (e.g., shortening occupancies, or delaying them by more than 1 time unit), rather than to keep adding dependency edges of the same kind. *Tightness* points to a minimal rewritable set: define minimality for a fixed repair capability and quantify the gap between the impact set and it. This is a necessary step

to advance "bounded" from a sufficient condition to a tight bound; the current impact set covers about 92% of the unfinished occupancies, so the gap may be considerable. *Size characterization* currently has only one post hoc rule (the rewritable share, Figure 8), but it varies with the schedule, and an indicator computable before the disturbance is needed instead. Section 6.6 has ruled out the family of static cut indicators: they are constant on practical layouts, whereas on the same network merely changing the machine configuration changes the impact-set share. On the 4×4 network, going from 4 to 5 machines changes it by 0.207; on the 5×5 network, the three configurations (6, 7, and 8 machines) differ by 0.075. This suggests that what matters may be the *spatial distribution of demand*, rather than the traffic capacity the network provides. The next candidate is the temporal density of corridor occupancies in the neighborhood of t_{now}, for example the number of overlapping occupancies on the disturbed corridor within its busy window, and the out-degree of that corridor in the yielding graph. Such quantities vary with the schedule and are no longer purely topological, so the notion of "predictability" must accordingly be restated as predicting the size from a given layout *and the current schedule*.

3. Relaxing Assumption A3 (Limitation 3; A3 stipulates a single, fully observed disturbance). The next step is not to open up resequencing further but to relax this assumption, examining whether the impact set accumulates and inflates under continual disturbances, and whether an explicit "restoration" step is needed between two repairs.

4. Taking the time calibration from an external source as well (Limitation 4). The layouts already come from an external source; for the time calibration, an instance family that publishes both *a network of exclusive corridors and per-leg durations* is still needed, and a self-built sweep over edge weights cannot substitute for it. A second independent implementation need not be undertaken separately; it can be carried out together with reproducing any of the items above.

Acknowledgments

This work was supported by <TO BE FILLED: funding program name and grant number>. The authors thank the anonymous reviewers for their comments on the experimental measures and the boundaries of the conclusions.

Conflict of interest statement

The authors declare no conflicts of interest related to this work.

References

[1] Riyad J. Abumaizar and Joseph A. Svestka. 1997. Rescheduling job shops under random disruptions. *International Journal of Production Research* 35, 7 (1997), 2065–2082. doi:10.1080/002075497195074

[2] M. Selim Aktürk and Elif Görgülü. 1999. Match-up scheduling under a machine breakdown. *European Journal of Operational Research* 112, 1 (1999), 81–97. doi:10.1016/S0377-2217(97)00396-2

[3] Ümit Bilge and Gündüz Ulusoy. 1995. A Time Window Approach to Simultaneous Scheduling of Machines and Material Handling System in an FMS. *Operations Research* 43, 6 (1995), 1058–1070. doi:10.1287/opre.43.6.1058

[4] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034

[5] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production–Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.

[6] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6

[7] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[8] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, Honolulu, HI, USA, 79–86. doi:10.1109/SCIS.2007.367673

[9] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[10] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109

[11] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Abu Dhabi, United Arab Emirates, 10214–10221. doi:10.1109/IROS58592.2024.10801975

[12] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1

[13] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[14] Djamila Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8

[15] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[16] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9-10 (2003), 669–680. doi:10.1007/s00170-003-1601-6

[17] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[18] Jiří Švancara, Marek Vlk, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. AAAI Press, Honolulu, HI, USA, 7732–7739. doi:10.1609/aaai.v33i01.33017732

[19] Hongtao Tang, Xuewei Zhang, Jiawei Huang, Xuesong Xu, and Jiansha Lu. 2026. Integrated Scheduling of Multi-Objective Lot-Streaming Hybrid Flowshop with AGV under Dynamic Environments. *International Journal of Industrial Engineering Computations* 17, 2 (2026), 295–316. doi:10.5267/j.ijiec.2025.9.003

[20] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[21] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958

[22] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[23] Yannik Zeiträg and José Rui Figueira. 2025. A novel machine-learning rolling horizon heuristic for dynamic lot-sizing and job shop scheduling problems. *International Journal of Production Research* 63, 12 (2025), 4563–4589.

[24] Hongliang Zhang, Chaoqun Qin, Wenhui Zhang, Zhenxing Xu, Gongjie Xu, and Zhenhua Gao. 2023. Energy-Saving Scheduling for Flexible Job Shop Problem with AGV Transportation Considering Emergencies. *Systems* 11, 2 (2023), 103. doi:10.3390/systems11020103

[25] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, Kate Larson (Ed.). International Joint Conferences on Artificial Intelligence Organization, Jeju, South Korea, 311–320. doi:10.24963/ijcai.2024/35

[26] Ziyan Zhao, Shixin Liu, MengChu Zhou, Xiwang Guo, and Liang Qi. 2019. Decomposition method for new single-machine scheduling problems from steel production systems. *IEEE Transactions on Automation Science and Engineering* 17, 3 (2019), 1376–1387.